Tecnológico Nacional de México

Campus Ciudad Juárez

Ingeniería en Sistemas Computacionales

Inteligencia Artificial

U2. Representación del conocimiento razonamiento y los Aspectos Metodológicos en Inteligencia Artificial.

Practicas

Grupo M

Nombre de los alumnos:

José Enrique Esquivel Torres

Irvin Eduardo Trujillo Lopez

Kenneth Eduardo Lara Hernandez

Carlos Daniel Perez Noriega

Nombre del docente:

Zapien Garcia Nancy Teresa

Fecha: 02/10/2026

APLICACION DE UN MODELO DE MACHINE LEARNING DE REGRESION LINEAL

In [92]:
```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call
drive.mount("/content/drive", force_remount=True).
```

En esta actividad usarás el archivo de datos `bigMartSales.csv`.

- `Item_Identifier`: ID de producto único
- `Item_Weight`: Peso del producto
- `Item_Fat_Content`: si el producto es bajo en grasa o no
- `Item_Visibility`: el porcentaje del área de exhibición total de todos los productos en una tienda asignado al producto en particular.
- `Item_Type`: La categoría a la que pertenece el producto.
- `Item_MRP`: Precio de venta máximo (precio de lista) del producto
- `Outlet_Identifier`: ID de tienda única
- `Outlet_Establishment_Year`: El año en que se estableció la tienda.
- `Outlet_Size`: El tamaño de la tienda en términos de superficie cubierta.
- `Outlet_Location_Type`: El tipo de ciudad en la que se encuentra la tienda.
- `Outlet_Type`: si el outlet es solo una tienda de comestibles o algún tipo de supermercado.
- **`Item_Outlet_Sales`**: Ventas del producto en la tienda concreta. Ésta es la variable de resultado que se va a predecir.

In [93]:
```
# Librerías para manipulación de datos
import numpy as np
import pandas as pd

# Librerías para visualización
import seaborn as sns
import matplotlib.pyplot as plt


# Librerías para preprocesamiento e ing de características
from sklearn.impute import SimpleImputer #sirve para rellenar los datos faltantes
(imputar)
                    #Cuando tienes datos con valores nulos (NaN), permite remplazarlos
con mean, median, most_frecuent, constant)
```

```
from sklearn.preprocessing import MinMaxScaler, OneHotEncoder
             #MinMaxScaler, normaliza los datos numericos dentro de un rango [0,1]
                   #ayuda a que los modelos ML funcionen mejor, reduce la influencia
de variables con escalas muy diferentes

             #OneHOTEncoder, convierte variables categoricas en una representacion
binaria
                   #Algoritmo de ML no entienden texto. Este enconder convierte cada
columna Categorica en na BInaria (0 y 1)

# Librerías para la canalización (pipeline)
    #es una forma ordenada de Encadenar pasos de preprocesamiento y modelado, forma
que puedas tratar tus datos y entrenar un modelo
    #todo en una sola estructura, sin repetir codigo ni cometer errores por hacer
pasos manuales

from sklearn.compose import ColumnTransformer, make_column_selector
      #ColumnTransformer, aplica diferentes transformaciones a diferentes columnas de
tu conjunto de datos
          #dado que NO TODAS las columnas necesitan el mismo tratamiento
          #Columnas numericas, se escalan con MinMAxScaler
          #Columnas categoricas se codifican con OneHOtEncoder

      #make_column_selector, selecciona automaticamente columnas numericas o
categoricas del DF segun su tipo de datos
          #Para no escribir los nombres de las columnas manualmente. Especialmente
util cuando tenemos muchas columnas y
          #queremos seleccionar todas las numericas o todas las categoricas sin
necesidad de enlistarlas

from sklearn.pipeline import make_pipeline
      #permite encadenar los pasos secuenciales de transformacion y modelado (Escalado
- Codificacion - modelado)
      #Se evita tener que hacer transformaciones por separado y nos asegura que se
aplicacn en el orden correcto, tanto al entrenar como al hacer predicciones


# Librerías para la regresión
from sklearn.linear_model import LinearRegression
      #modelo de Regresion Lineal clasico de Scikit-learn
      #ENTRENA UN MODELO DE REGRESION LINEAL PARA PREDECIR VALORES NUMERICOS
      #se utiliza para predecir valores numericos en funcion de una variable o mas
variables independientes.
      #Busca una linea recta que mejor se ajuste a los datos

from sklearn.metrics import mean_squared_error, mean_squared_error, r2_score
      #mean_squared_error(MSE), la media de los cuadrados de las diferencias entre los
valores reales y los predichos
      #EVALUA EL MODELO MIDIENDO EL ERROR CUADRATICO MEDIO
      #Cuanto mas bajo el MSE, mejor es el modelo
      #tienen las mismas unidades que la variable objetivo al elevar al cuadrado, lo
que penaliza los errores

      #Coeficiente de determinacion (r2_score), indica que la proporcion de la
varianza en la variable dependiente se puede explciar por las variables independientes
```

```
#EVALUA EL MODELO INDICANDO QUE TAMBIEN EXPLICA LA VARIACION DE LOS DATOS
#su valor va entre 0 y 1
#r2_score=1, el modelo predice de manera perfecta
#r2_scores=0, el modelo no explica nada mejor que una media
```

Descarga el archivo: `bigMartSales.csv` y guarda, en un dataframe (`sales_df`), todos sus registros.

In [94]:
```
sales_df = pd.read_csv('/content/drive/MyDrive/NUBE/bigMartSales.csv')
sales_df
```

Out[94]:

	Item_Identifier	Item_Weight	Item_Fat_Content	Item_Visibility	Item_Type	Item_MRP	Outlet_Identifier	Outlet_Establishment_Year	Outlet_Size	Outle
0	FDA15	9.300	Low Fat	0.016047	Dairy	249.8092	OUT049	1999	Medium	
1	DRC01	5.920	Regular	0.019278	Soft Drinks	48.2692	OUT018	2009	Medium	
2	FDN15	17.500	Low Fat	0.016760	Meat	141.6180	OUT049	1999	Medium	
3	FDX07	19.200	Regular	0.000000	Fruits and Vegetables	182.0950	OUT010	1998	NaN	
4	NCD19	8.930	Low Fat	0.000000	Household	53.8614	OUT013	1987	High	
...	...	...	...	...	...	...	...	...	...	
8518	FDF22	6.865	Low Fat	0.056783	Snack Foods	214.5218	OUT013	1987	High	
8519	FDS36	8.380	Regular	0.046982	Baking Goods	108.1570	OUT045	2002	NaN	
8520	NCJ29	10.600	Low Fat	0.035186	Health and Hygiene	85.1224	OUT035	2004	Small	
8521	FDN46	7.210	Regular	0.145221	Snack Foods	103.1332	OUT018	2009	Medium	
8522	DRG01	14.800	Low Fat	0.044878	Soft Drinks	75.4670	OUT046	1997	Small	

8523 rows × 12 columns

ANALISIS DE DATOS

In [95]:
```
sales_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8523 entries, 0 to 8522
Data columns (total 12 columns):
 #   Column                     Non-Null Count  Dtype
---  ------                     --------------  -----
 0   Item_Identifier            8523 non-null   object
 1   Item_Weight                7060 non-null   float64
 2   Item_Fat_Content           8523 non-null   object
 3   Item_Visibility            8523 non-null   float64
 4   Item_Type                  8523 non-null   object
 5   Item_MRP                   8523 non-null   float64
 6   Outlet_Identifier          8523 non-null   object
 7   Outlet_Establishment_Year  8523 non-null   int64
 8   Outlet_Size                6113 non-null   object
 9   Outlet_Location_Type       8523 non-null   object
 10  Outlet_Type                8523 non-null   object
 11  Item_Outlet_Sales          8523 non-null   float64
dtypes: float64(4), int64(1), object(7)
memory usage: 799.2+ KB
```

In [96]:
```
# Determinando la cantidad de valores únicos por columna
sales_df.nunique()
```

Out[96]:

	0
Item_Identifier	1559
Item_Weight	415
Item_Fat_Content	5
Item_Visibility	7880
Item_Type	16
Item_MRP	5938
Outlet_Identifier	10
Outlet_Establishment_Year	9
Outlet_Size	3
Outlet_Location_Type	3
Outlet_Type	4
Item_Outlet_Sales	3493

dtype: int64

Como las variables `Item_Identifier` y `Outlet_Identifier` son solo variables de ID, asumimos que no tienen ningún poder predictivo para predecir la variable dependiente: `Item_Outlet_Sales`.

In [97]:
```
sales_df.drop(['Item_Identifier','Outlet_Identifier'], axis=1, inplace=True)
```

In [98]:
```
# Estadísticas descriptivas de las variables numéricas
sales_df.describe().T
```

Out[98]:

	count	mean	std	min	25%	50%	75%	max
Item_Weight	7060.0	12.857645	4.643456	4.555	8.773750	12.600000	16.850000	21.350000
Item_Visibility	8523.0	0.066132	0.051598	0.000	0.026989	0.053931	0.094585	0.328391
Item_MRP	8523.0	140.992782	62.275067	31.290	93.826500	143.012800	185.643700	266.888400
Outlet_Establishment_Year	8523.0	1997.831867	8.371760	1985.000	1987.000000	1999.000000	2004.000000	2009.000000
Item_Outlet_Sales	8523.0	2181.288914	1706.499616	33.290	834.247400	1794.331000	3101.296400	13086.964800

In [99]:
```
# Estadísticas descriptivas de las variables de texto
sales_df.describe(include = 'object').T
```

Out[99]:

	count	unique	top	freq
Item_Fat_Content	8523	5	Low Fat	5089
Item_Type	8523	16	Fruits and Vegetables	1232
Outlet_Size	6113	3	Medium	2793
Outlet_Location_Type	8523	3	Tier 3	3350
Outlet_Type	8523	4	Supermarket Type1	5577

In [100…
```
# Se hace una lista de variables numéricas y otra de categóricas.
num_cols = sales_df.select_dtypes(include=np.number).columns.tolist()
cat_cols = sales_df.select_dtypes(exclude=np.number).columns.tolist()
```

In [101…
```
# Recuento de cada categoría única en las variables categóricas
for column in cat_cols:
    print(column)
    print(sales_df[column].value_counts())
    print('-' * 50)
```

```
Item_Fat_Content
Item_Fat_Content
Low Fat    5089
Regular    2889
LF          316
reg         117
low fat     112
Name: count, dtype: int64
--------------------------------------------------
Item_Type
Item_Type
Fruits and Vegetables    1232
Snack Foods              1200
Household                 910
Frozen Foods              856
Dairy                     682
Canned                    649
Baking Goods              648
Health and Hygiene        520
Soft Drinks               445
Meat                      425
Breads                    251
Hard Drinks               214
Others                    169
Starchy Foods             148
Breakfast                 110
Seafood                    64
Name: count, dtype: int64
--------------------------------------------------
Outlet_Size
Outlet_Size
Medium    2793
Small     2388
High       932
Name: count, dtype: int64
--------------------------------------------------
Outlet_Location_Type
Outlet_Location_Type
Tier 3    3350
Tier 2    2785
Tier 1    2388
Name: count, dtype: int64
--------------------------------------------------
Outlet_Type
Outlet_Type
Supermarket Type1    5577
Grocery Store        1083
Supermarket Type3     935
Supermarket Type2     928
Name: count, dtype: int64
--------------------------------------------------
```

In [102…

```python
# Se determinan porcentaje de nulos por columna
sales_df.isnull().mean() * 100
```

Out[102…

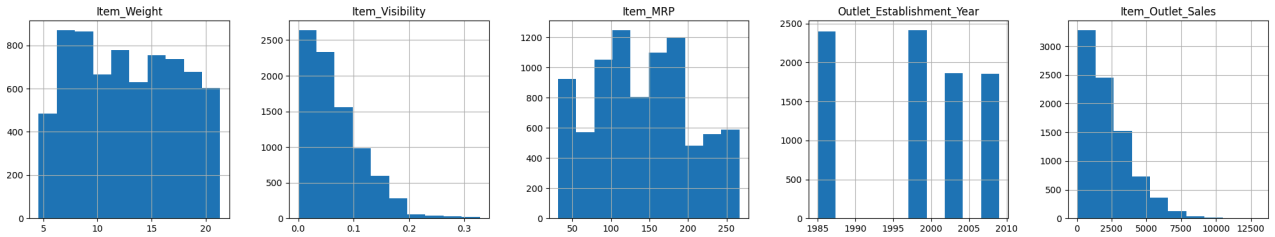
	0
Item_Weight	17.165317
Item_Fat_Content	0.000000
Item_Visibility	0.000000
Item_Type	0.000000
Item_MRP	0.000000
Outlet_Establishment_Year	0.000000
Outlet_Size	28.276428
Outlet_Location_Type	0.000000
Outlet_Type	0.000000
Item_Outlet_Sales	0.000000

dtype: float64

Análisis univariado de variables numéricas

In [103…

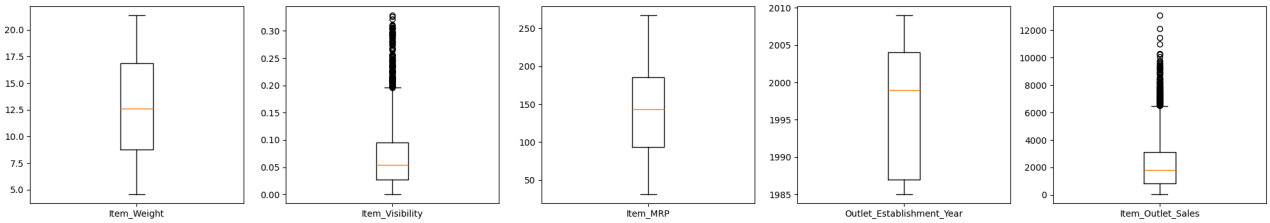
```
sales_df.hist(figsize=(25,4), layout=(1,5))
plt.show()
```



In [104…

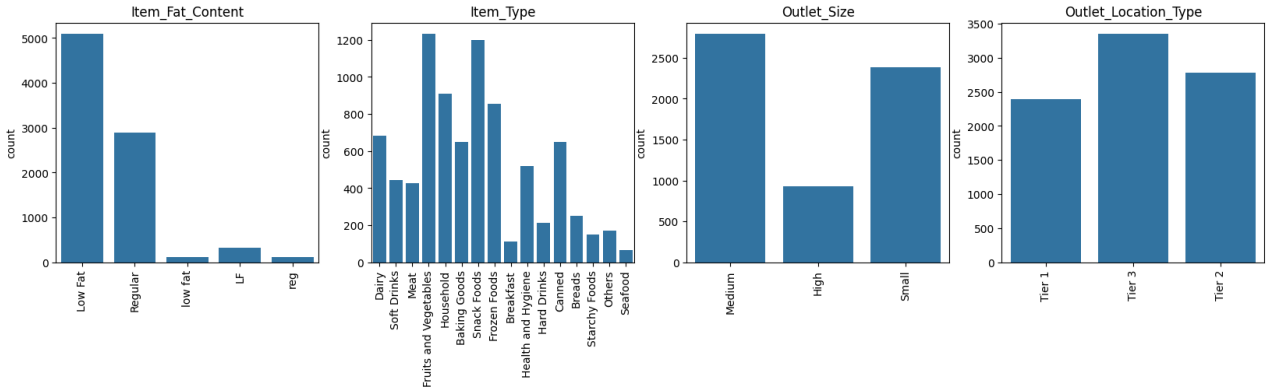
```
fig, axes = plt.subplots(1,5, figsize=(25,4))
axes = axes.ravel()
for col, ax in zip(sales_df[num_cols], axes):
  ax.boxplot(sales_df[col].dropna(), labels=[f'{col}'])
```

```
/tmp/ipykernel_2518/1984969689.py:4: MatplotlibDeprecationWarning: The 'labels'
parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for
the old name will be dropped in 3.11.
  ax.boxplot(sales_df[col].dropna(), labels=[f'{col}'])
/tmp/ipykernel_2518/1984969689.py:4: MatplotlibDeprecationWarning: The 'labels'
parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for
the old name will be dropped in 3.11.
  ax.boxplot(sales_df[col].dropna(), labels=[f'{col}'])
/tmp/ipykernel_2518/1984969689.py:4: MatplotlibDeprecationWarning: The 'labels'
parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for
the old name will be dropped in 3.11.
  ax.boxplot(sales_df[col].dropna(), labels=[f'{col}'])
/tmp/ipykernel_2518/1984969689.py:4: MatplotlibDeprecationWarning: The 'labels'
parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for
the old name will be dropped in 3.11.
  ax.boxplot(sales_df[col].dropna(), labels=[f'{col}'])
/tmp/ipykernel_2518/1984969689.py:4: MatplotlibDeprecationWarning: The 'labels'
parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for
the old name will be dropped in 3.11.
  ax.boxplot(sales_df[col].dropna(), labels=[f'{col}'])
```



Análisis univariado de variables categóricas

In [105…
```
fig, axes = plt.subplots(1,4, figsize=(20,4))
axes = axes.ravel()
for col, ax in zip(sales_df[cat_cols], axes):
  sns.countplot(x=sales_df[col], ax=ax)
  ax.set(title=f'{col}', xlabel=None)
  ax.tick_params(axis='x',rotation=90)
```



En `Ite_Fat_Content` hay categorías expresadas de diferente forma:

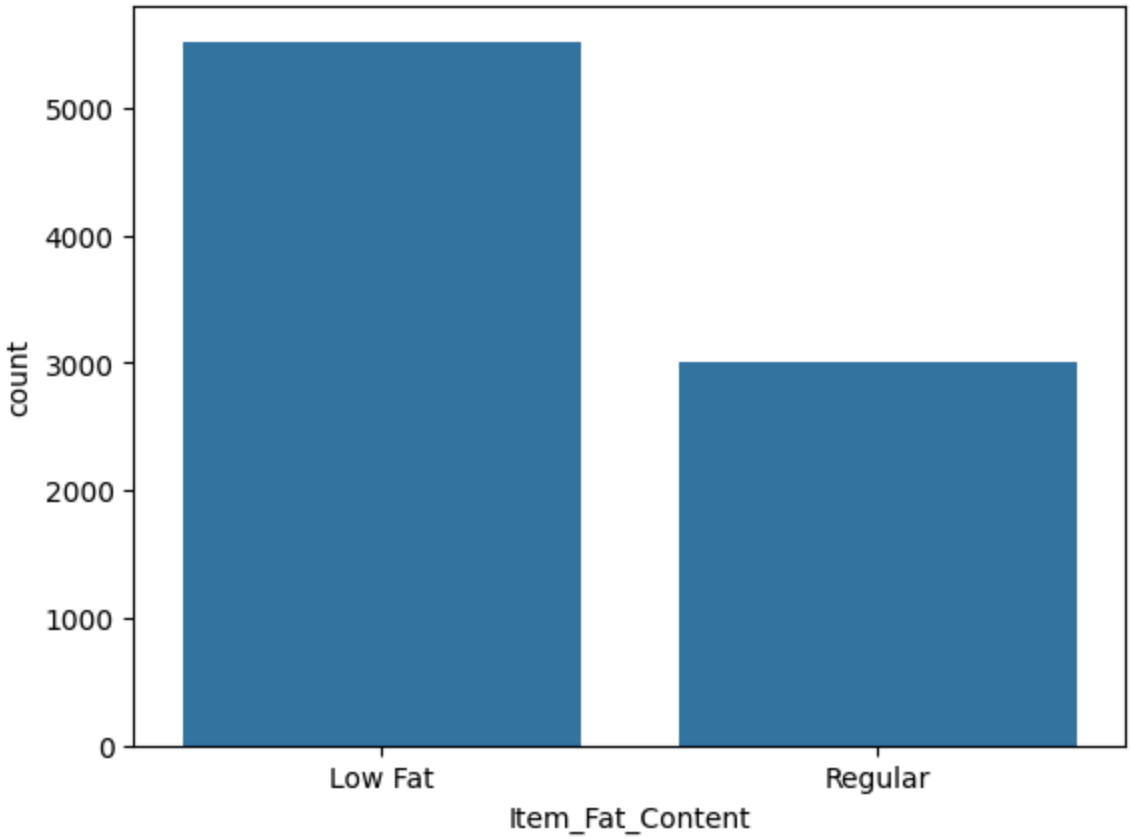
- Low Fat = low fat = LF
- Regular = reg

Se unifica, para sólo dejar dos valores: Low Fat y Regular

In [106…
```
sales_df['Item_Fat_Content'] = sales_df['Item_Fat_Content'].apply(lambda x: 'Low Fat'
if x == 'low fat' or x == 'LF' else x)
sales_df['Item_Fat_Content'] = sales_df['Item_Fat_Content'].apply(lambda x: 'Regular'
if x == 'reg' else x)
```

In [107…
```
sns.countplot(x=sales_df['Item_Fat_Content'])
```

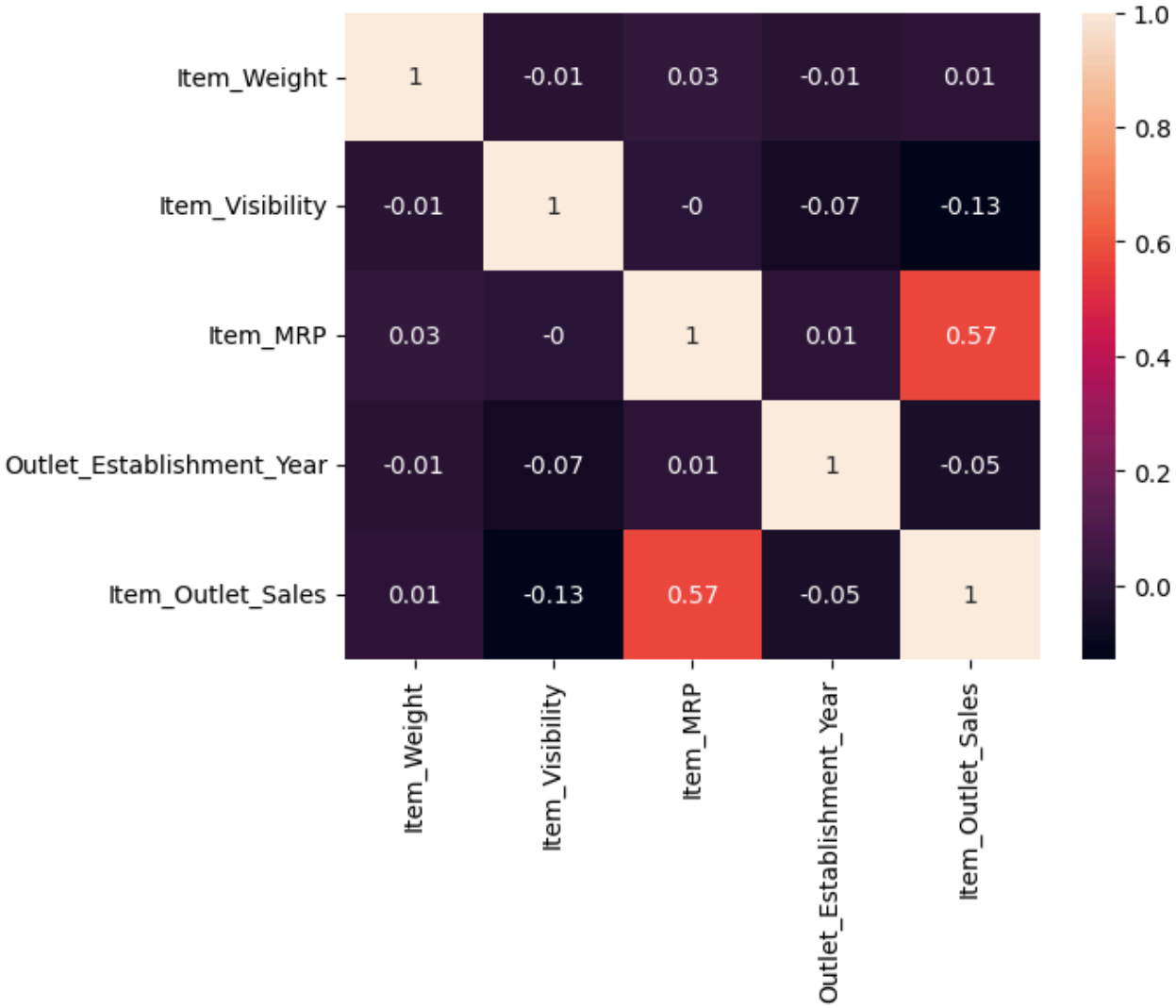
Out[107…
```
<Axes: xlabel='Item_Fat_Content', ylabel='count'>
```



Análisis bivariado

In [108…
```
sns.heatmap(round(sales_df.corr(numeric_only=True),2), annot = True)
```

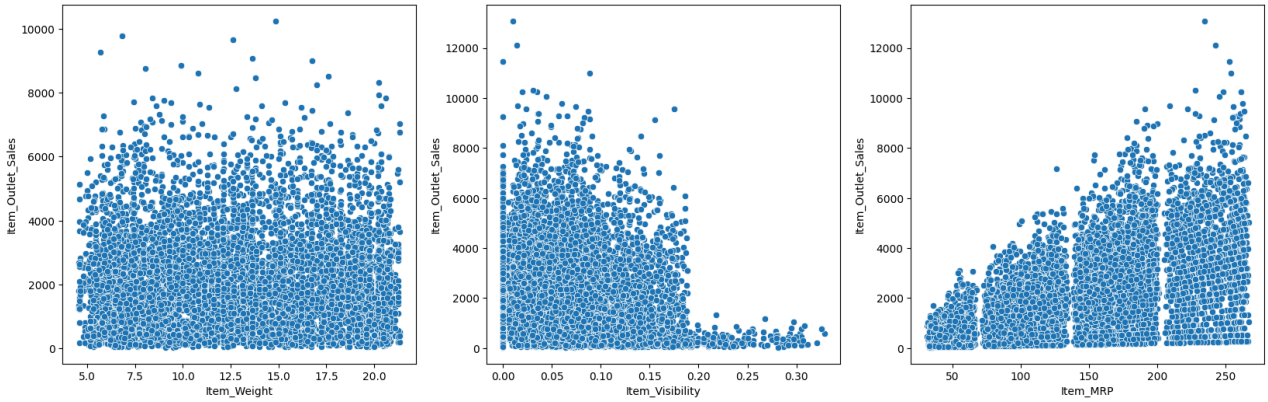
Out[108…
```
<Axes: >
```



Del gráfico anterior, parece que solo la variable independiente `Item_MRP` tiene una relación lineal moderada con la variable dependiente `Item_Outlet_Sales`. Se verifica con diagramas de dispersión.

In [109…
```
fig, axes = plt.subplots(1, 3, figsize = (20, 6))
sns.scatterplot(x='Item_Weight', y='Item_Outlet_Sales', data=sales_df, ax=axes[0])
sns.scatterplot(x='Item_Visibility', y='Item_Outlet_Sales', data=sales_df, ax=axes[1])
sns.scatterplot(x='Item_MRP', y='Item_Outlet_Sales', data=sales_df, ax=axes[2])
```

Out[109…
```
<Axes: xlabel='Item_MRP', ylabel='Item_Outlet_Sales'>
```



Se crea una nueva característica `Outlet_Age` que indica la antigüedad del outlet.

In [110…

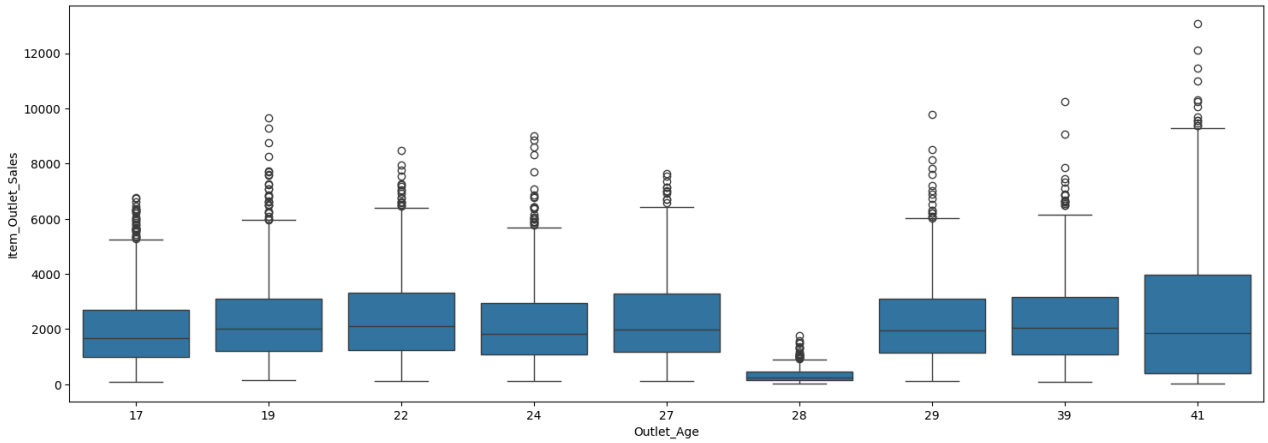
```
from datetime import date #permite trabajar con fechas
#date.today().year, obtiene el año actual
#sales_df['Outlet_Establishment_year'], columna que tinee el año que se fundo cada tienda
sales_df['Outlet_Age'] = date.today().year - sales_df['Outlet_Establishment_Year']
#calcula la edad de cada tienda en años

sales_df.drop('Outlet_Establishment_Year', axis = 1, inplace=True) #elimina la columan original
    #se elimina porque ya no es necesaria, ya que fue transformada auna forma mas util para el modelo
    #ayuda a reducir la dimensionalidad del dataset y evita redundancia

    #axis=1, indica que se elimnara una COLUMNA (en lugar de una fila)
    #inplace=true, guarda cambios directamente en el df
```

In [111…

```
fig = plt.figure(figsize = (18, 6))
sns.boxplot(x = 'Outlet_Age', y = 'Item_Outlet_Sales', data = sales_df);
```



INCIO MODELO MACHINE LEARNING

INGENIERIA DE CARACTERISTICAS

Se especifican cuales seran las variables de entrada 'X' (Caracteristica) y cual sera la varible predictora 'y' (variable salida o tambien Objetivo)

In [112…
```
# Se separan las variables del dataframe: en X quedan los predictores y en y la
variable de respuesta o salida (Item_Outlet_Sales).
X = sales_df.drop('Item_Outlet_Sales', axis = 1) #variables de entrada -
Caracteristica
y = sales_df['Item_Outlet_Sales'] #variable salida -variable predictora - Objetivo
```

PARTICION DEL CONJUNTO DE DATOS

In [113…
```
# Se divide el conjunto en entrenamiento y prueba (80:20)
#80% conjunto de entrenamiento(train)
#20% para el conjunto de prueba (test)

from sklearn.model_selection import train_test_split

Xtrain, Xtest, ytrain, ytest = train_test_split(X,#variable entrada
                                                y,#variable salida o predictora
                                                train_size=0.8, # tama;o conjunto
entrenamiento
                                                random_state=1) #para mantener mismo
conjunto de datos
```

PROCESAMIENTO

CANALIZACION (PIPELINE)

In [114…
```
# Como se aplicará más de un transformador y de manera diferenciada (numéricas y
categóricas reciben diferentes tratamientos),
# hay que crear pipelines (aquí se nombraron como: num_pipeline y cat_pipeline) para
transformar las columnas

# Si sólo fuera un transformador en cada tipo de variable, no sería necesario crer
num_pipeline ni cat_pipeline.
# El transformador único podría ir directamente en el ColumnTransformer que aparece
abajo
#PARA VARIABLES NUMERICAS
num_pipeline = make_pipeline(SimpleImputer(strategy='median'), MinMaxScaler())
      #Rellena valores faltantes con la media
      #aplica escalado MinMaxScaler, para que todas los valores numericos esten en el
rango [0,1]


#PARA VARIABLES CATEGORICAS
cat_pipeline = make_pipeline(SimpleImputer(strategy='most_frequent'),
OneHotEncoder(drop='first', handle_unknown='ignore', sparse_output=False))
      #Rellena valores faltantes con la categoria mas frecuente
      #Convierte categorias en columnas binarias (0,1) con OneHOt

        #parametros de OneHot
          #drop='first', evita la trampa multicolinealidad, eliminando la primer
categoria
                #Multicolaniedad- se refiere cuando se da la situacion que dos o mas
columnas en un modelo de regresion estan altamente correlacionadas entre si.
                    #es decir, que el modelo no distinga cual es 'y' (variable
```

```
predictora)
                    #esta alta correlacion puede generar problemas de ajuste del
modelo.
                    #si nos variables (columnas), son muy similares, el modelo no
puede distinguir cual de ellas tiene mayor influencia en X (variables de entrada)

          #handle_unknown='ignore', ignora nuevas categorias que no se vieron durante
el entrenamiento (evita errores de produccion)
          #sparse_output=False, devuelve matriz densa, util si vas a convertir el
resultado a un DF)

#UNE PREPROCESAMIENTOS
preprocessing = ColumnTransformer([
    ('num', num_pipeline, make_column_selector(dtype_include=np.number)), #detecta
TODAS columnas numericas
, make_column_selector(dtype_include=np.number)),
    ('cat', cat_pipeline, make_column_selector(dtype_include=object))])   #detecta
TODAS columnas categoricas
```

In [115…

```
# Esto no es necesario, pero lo incluyo para que se visualicen los cambios que produce
el pipeline "preprocessing" creado anteriormente
# Nótese que Xtrain queda con 27 columnas después del procesamiento, es decir, se
utilizarán 27 predictores en el modelo (4 numéricos y 23 categóricos)

Xtrain_prepared = preprocessing.fit_transform(Xtrain)
Xtrain_prepared_df = pd.DataFrame(Xtrain_prepared,
columns=preprocessing.get_feature_names_out(), index=Xtrain.index)
Xtrain_prepared_df
```

Out[115…

	num__Item_Weight	num__Item_Visibility	num__Item_MRP	num__Outlet_Age	cat__Item_Fat_Content_Regular	cat__Item_Type_Breads	cat__Item_Type_Bre
1945	0.821375	0.272069	0.681925	0.000000	0.0	0.0	
1720	0.761834	0.511783	0.615374	0.000000	0.0	0.0	
1954	0.330158	0.164095	0.824788	0.083333	1.0	0.0	
1919	0.374814	0.496066	0.312702	0.000000	0.0	0.0	
2461	0.155701	0.181846	0.423207	0.208333	1.0	0.0	
...	...	...	...	...	...	...	
2895	0.479012	0.844906	0.531008	1.000000	1.0	0.0	
7813	0.479012	0.542624	0.100619	1.000000	0.0	0.0	
905	0.791605	0.200402	0.500067	0.500000	0.0	0.0	
5192	0.300387	0.151094	0.054833	0.458333	0.0	0.0	
235	0.479012	0.132688	0.685890	1.000000	0.0	0.0	

6818 rows × 27 columns

APLICA PREPROCESAMIENTO ANTERIOR PARA EL

ENTRENAMIENTO DEL MODELO REGRESION LINEAL

In [116…

```
# Se crea el pipeline para la predicción, que incluye el de procesamiento creado
anteriormente y la regresión lineal

#CREA PIPELINE
```

```
lr_model = make_pipeline(preprocessing, LinearRegression())
    #PREPROCESING, procesa los datos (relleno de nulos, escalado (minMaxScaler),
codificacion (OneHOtEnconder))
    #LinearRegresion, aplica modelo de regresion lineal a los datos ya entrenados

#ENTRENA Y AJUSTA TODO EL PIPELINE CON LOS DATOS DE ENTRENAMIENTO
lr_model.fit(Xtrain, ytrain)
    #primero transforma Xtrain con el preprocesador
    #luego entrena el modelo de LinearRegresion() con las variables transformadas y la
variable objetivo ytrain

#APLICA PIPELINE A LOS DATOS DE PRUEBA
predictions = lr_model.predict(Xtest)
  #primero transforma con el mismo preprocesador (usando los parametros aprendidos en
fit)
  #despues realiza predicciones con el modelo entrenado

#CALCULA VALOR RMSE ENTRE VALORES REALES (ytest) y los valores predichos por el modelo
(Predictions)
# Línea corregida usando numpy
import numpy as np
print('The Root Mean Square Error (RMSE) is:', np.sqrt(mean_squared_error(ytest,
predictions)))
#RMSE nos dice en promedio cuanto se equivoco tu modelo al predecir, en las mismas
unidades que la variable objetivo

#esta metrica indica que proporcion de la variacion de la variable dependiente (ytest)
es explicada por el modelo
print('The R square (R2) is:', r2_score(ytest, predictions))
#r2= 1, prediccion perfecta
#r2= 0, el modelo no explica mejor que un promedio
#r2<0, el modelo es peor
```

```
The Root Mean Square Error (RMSE) is: 1144.3827848578724
The R square (R2) is: 0.5582080574611583
```

RMSE = 1144.38, significa que el modelo se equivoca por unas 1144.38 unidades.

r2 =0.558, significa que elmodelo explcia el 55.8% de la variabilidad de los datos, el 42% restante, no lo puede explicar (puede deberse al ruido, datos faltantes, variables importantes no incluidas), o hacer uso de otro tipo de transformaciones de variables.

El resultado obtenido por el modelo de regresion lineal no es excelente, debido a que esta haciendo predicciones con un error moderado y explica poco mas de la mitad del comportamiento de la variable objetivo

Lo que seguiría es hacer más transformaciones para mejorar el resultado. Por ejemplo:

- Aplicar una transformación (log, box-cox,...) para hacer que la distribución de `Item_Visibility` tenga una forma más normal, porque está sesgada.
- Usar StandarScaler o RobustScaler en lugar de MinMax.

- Aplicar OrdinalEncoder a algunas categóricas, en lugar de OneHotEncoder.

In [117…
```
# Se puede tener acceso a los pasos ejecutados en el pipeline
lr_model.named_steps
```

Out[117…
```
{'columntransformer': ColumnTransformer(transformers=[('num',
                                 Pipeline(steps=[('simpleimputer',
                                                  SimpleImputer(strategy='median')),
                                                 ('minmaxscaler',
                                                  MinMaxScaler())]),

<sklearn.compose._column_transformer.make_column_selector object at 0x7ff0c6487680>),
                                ('cat',
                                 Pipeline(steps=[('simpleimputer',

SimpleImputer(strategy='most_frequent')),
                                                 ('onehotencoder',
                                                  OneHotEncoder(drop='first',

handle_unknown='ignore',

sparse_output=False))]),

<sklearn.compose._column_transformer.make_column_selector object at
0x7ff0c19ad8c0>)]),
 'linearregression': LinearRegression()}
```

In [118…
```
# Una vez ubicado en un paso, por ejemplo en la regresión lineal ('linearregression'),
se puede tener acceso a sus atributos
# Para mostrar la intercepción o beta_0
lr_model.named_steps['linearregression'].intercept_
```

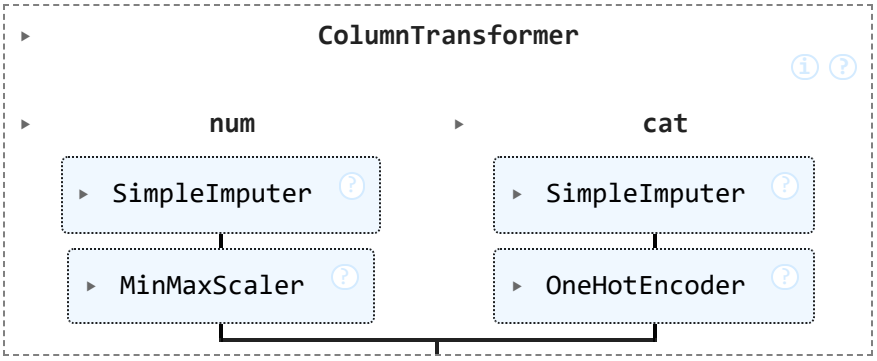
Out[118…
```
np.float64(274.6134468529333)
```

In [119…
```
# Para mostrar los coeficientes: beta_1, beta_2, ... beta_n
# En este caso se obtienen 27 coeficientes, uno por predictor
lr_model.named_steps['linearregression'].coef_
```

Out[119…
```
array([  28.62355263,  -97.09612341, 3678.93686232, -883.7484864 ,
          36.0930557 ,    9.09628309,  -15.7209235 ,   34.68740649,
       -100.45813405,  -39.04913619,   38.07938416,  -23.89284596,
         10.9988591 ,  -62.51983653,  -33.90791199,  -36.06410826,
        283.24072408,   15.10818476,  -74.74109576,   35.98874243,
       -839.68268244, -746.65228891, -202.1755319 , -395.31896407,
       1509.68985198, 1243.4315047 , 3872.29804368])
```

In [120…
```
# Para mostrar los transformadores ejecutados
lr_model.named_steps['columntransformer']
```

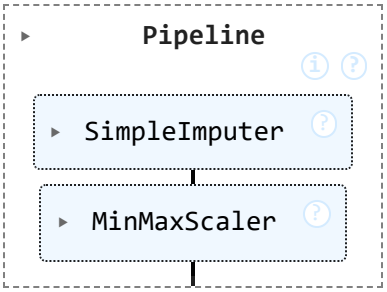
Out[120…



In [121…

```
# Para tener acceso a un transformador en particular
lr_model.named_steps['columntransformer'].named_transformers_['num']
```

Out[121…



In [122…

```
# Para tener acceso a los atributos de un transformador en particular
# Como sólo se tienen 4 predictores numéricos, se calcula el mínimo de cada uno
lr_model.named_steps['columntransformer'].named_transformers_['num'].named_steps['minm
axscaler'].min_
```

Out[122…

```
array([-0.27121167,  0.        , -0.13281075, -0.70833333])
```

TAREA: ALCANZAR UN 70 EN (R2)

In [126…

```
import numpy as np
import pandas as pd

# 1. Corregir categorías mal escritas en Item_Fat_Content (si aún existen)
if 'Item_Fat_Content' in sales_df.columns:
    sales_df['Item_Fat_Content'] = sales_df['Item_Fat_Content'].replace({
        'LF': 'Low Fat',
        'low fat': 'Low Fat',
        'reg': 'Regular'
    })

# 2. Corregir la visibilidad en cero usando la mediana por tipo de producto
if 'Item_Visibility' in sales_df.columns and 'Item_Type' in sales_df.columns:
    sales_df['Item_Visibility'] = sales_df['Item_Visibility'].replace(0, np.nan)
    sales_df['Item_Visibility'] = sales_df.groupby('Item_Type')
['Item_Visibility'].transform(lambda x: x.fillna(x.median()))

    # 3. Crear característica: Visibilidad Relativa del Producto
    type_vis_avg = sales_df.groupby('Item_Type')['Item_Visibility'].mean()
    sales_df['Item_Visibility_Ratio'] = sales_df.apply(
        lambda x: x['Item_Visibility'] / type_vis_avg[x['Item_Type']], axis=1
```

```
    )

# 4. Calcular Antigüedad de la Tienda (Outlet_Years) solo si existe el año y no se ha creado
if 'Outlet_Establishment_Year' in sales_df.columns:
    sales_df['Outlet_Years'] = 2013 - sales_df['Outlet_Establishment_Year']
    # 5. Crear el DataFrame definitivo excluyendo la columna redundante
    df_model = sales_df.drop(['Outlet_Establishment_Year'], axis=1)
else:
    df_model = sales_df.copy()

print("--- INGENIERÍA DE CARACTERÍSTICAS COMPLETADA ---")
print("Total de columnas para el modelo:", df_model.shape[1])
display(df_model.head())
```

```
--- INGENIERÍA DE CARACTERÍSTICAS COMPLETADA ---
Total de columnas para el modelo: 11
```

	Item_Weight	Item_Fat_Content	Item_Visibility	Item_Type	Item_MRP	Outlet_Size	Outlet_Location_Type	Outlet_Type	Item_Outlet_Sales	Outlet_Age	Item_
0	9.30	Low Fat	0.016047	Dairy	249.8092	Medium	Tier 1	Supermarket Type1	3735.1380	27	
1	5.92	Regular	0.019278	Soft Drinks	48.2692	Medium	Tier 3	Supermarket Type2	443.4228	17	
2	17.50	Low Fat	0.016760	Meat	141.6180	Medium	Tier 1	Supermarket Type1	2097.2700	27	
3	19.20	Regular	0.058778	Fruits and Vegetables	182.0950	NaN	Tier 3	Grocery Store	732.3800	28	
4	8.93	Low Fat	0.049017	Household	53.8614	High	Tier 3	Supermarket Type1	994.7052	39	

In [127…

```
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

# 1. Separar características (X) y variable objetivo (y)
X = df_model.drop('Item_Outlet_Sales', axis=1)
y = df_model['Item_Outlet_Sales']

# 2. Transformación logarítmica para reducir la asimetría de las ventas
y_log = np.log1p(y)

# 3. Dividir el dataset en entrenamiento (80%) y prueba (20%)
X_train, X_test, y_train_log, y_test_log = train_test_split(X, y_log, test_size=0.2, random_state=42)

# Conservar y_test en valores reales para evaluar al final
_, _, _, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 4. Identificar columnas numéricas y categóricas
num_cols = X.select_dtypes(include=[np.number]).columns.tolist()
cat_cols = X.select_dtypes(exclude=[np.number]).columns.tolist()

# 5. Definir Pipeline de preprocesamiento
num_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
```

```
    ('scaler', StandardScaler())
])

cat_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
])

preprocessor = ColumnTransformer([
    ('num', num_pipeline, num_cols),
    ('cat', cat_pipeline, cat_cols)
])

print("--- DIVISIÓN Y PREPROCESAMIENTO CONFIGURADO ---")
print(f"Registros de entrenamiento: {X_train.shape[0]} | Registros de prueba:
{X_test.shape[0]}")
```

```
--- DIVISIÓN Y PREPROCESAMIENTO CONFIGURADO ---
Registros de entrenamiento: 6818 | Registros de prueba: 1705
```

In [133…

```
from sklearn.pipeline import Pipeline
from xgboost import XGBRegressor
from sklearn.metrics import r2_score, mean_squared_error
import numpy as np

# 1. Definir el modelo no lineal (XGBoost) dentro del Pipeline
model_xgb = Pipeline([
    ('preprocessor', preprocessor),
    ('regressor', XGBRegressor(
        n_estimators=150,
        learning_rate=0.03,
        max_depth=5,
        subsample=0.8,
        colsample_bytree=0.8,
        random_state=42
    ))
])

# 2. Entrenar el modelo
model_xgb.fit(X_train, y_train_log)

# 3. Predecir y revertir la escala logarítmica (np.expm1)
preds_log = model_xgb.predict(X_test)
preds_real = np.expm1(preds_log)

# 4. Calcular métricas finales
r2 = r2_score(y_test, preds_real)
rmse = np.sqrt(mean_squared_error(y_test, preds_real))

print("========================================")
print("          RESULTADOS DEL MODELO MEJORADO          ")
print("========================================")
print(f"R² Score: {r2:.4f} ")
print(f"RMSE:     ${rmse:.2f}")
print("========================================")
```

```
========================================
          RESULTADOS DEL MODELO MEJORADO
========================================
R² Score: 0.5874
RMSE:     $1058.98
========================================
```

CONCLUSION

Tras multiples procesos y compilaciones, R2 se queda estancado en un rango de 0.58 a 0.60, sin poder avanzar mas, a pesar de el uso de caracteristicas y modelos de ensamblaje avanzados como XGBoost. No por error de codigo, sino por que el mismo DataSet proporcionado tiene un limite. La informacion no incluye detalles mas alla de lo establecido, al igual que el comportamiento que tiene un grado de caos que ningun algoritmo puede adivinar por los datos faltantes.

Curso: Inteligencia Artificial

UNIDAD 2

- NOMBRE: Jose Enrique Esquivel Torres
- NO CONTROL: 23110182

In [ ]:
```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

In [ ]:
```
import os
DIR = "/content/drive/MyDrive/Colab Notebooks/Analitica_De_Datos/U2/Ejemplos/data.csv"
```

In [ ]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Librerías para preprocesamiento e ing de características
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import MinMaxScaler, OneHotEncoder,StandardScaler,
OrdinalEncoder,FunctionTransformer

# Librerías para la canalización
from sklearn.compose import ColumnTransformer, make_column_selector
from sklearn.pipeline import make_pipeline

# Librerías para la regresión
from sklearn.linear_model import LinearRegression
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.metrics import mean_squared_error, mean_squared_error, r2_score,
confusion_matrix, precision_score,recall_score,accuracy_score
from sklearn.decomposition import PCA

from sklearn.preprocessing import PolynomialFeatures
```

Explica brevemente el uso de las librerias del segmento de codigo anterior, haciendo uso de la siguiente tabla

---

Library	Explanation
import pandas as pd	Se utiliza para manipular y organizar estructuras de datos en tablas.
import numpy as np	Se utiliza para realizar operaciones matemáticas y cálculos numéricos avanzados.
import matplotlib.pyplot as plt	Sirve para crear visualizaciones básicas como gráficas estadísticas y diagramas.

Library	Explanation
import seaborn as sns	Es una extensión de matplotlib, permite crear visualizaciones bonitas con menos código.
from sklearn.impute import SimpleImputer	Se utiliza para tratar los datos faltantes o nulos, reemplazándolos por la media, mediana, moda, etc.
from sklearn.preprocessing import MinMaxScaler	Escala los datos numéricos para que queden dentro de un rango específico, comúnmente entre 0 y 1.
from sklearn.preprocessing import OneHotEncoder	Convierte variables categóricas de texto a un formato numérico binario de unos y ceros.
from sklearn.preprocessing import StandardScaler	Estandariza los datos numéricos para que tengan una media de 0 y una desviación estándar de 1.
from sklearn.preprocessing import OrdinalEncoder	Convierte categorías de texto con un orden específico en valores numéricos ordenados (0, 1, 2, etc.).
from sklearn.preprocessing import FunctionTransformer	Permite aplicar funciones matemáticas personalizadas o creadas por el usuario a las columnas.
from sklearn.compose import ColumnTransformer	Permite aplicar diferentes transformaciones a columnas específicas del conjunto de datos.
from sklearn.compose import make_column_selector	Selecciona automáticamente columnas de un dataset basándose en su tipo de dato (numérico o texto).
from sklearn.pipeline import make_pipeline	Permite agrupar secuencialmente los pasos de preprocesamiento y el modelo en un solo flujo.
from sklearn.linear_model import LinearRegression	Modelo de aprendizaje supervisado que se usa para predecir valores numéricos continuos.
from sklearn.linear_model import LogisticRegression	Modelo matemático que sirve para clasificar y tomar decisiones de 'Sí o No' (binario).
from sklearn.metrics import mean_squared_error	Mide el error promedio entre los valores reales y las predicciones del modelo de regresión.
from sklearn.metrics import r2_score	Indica qué tan bien se ajustan las predicciones del modelo de regresión a los datos reales (0 a 1).
from sklearn.metrics import confusion_matrix	Muestra el resumen de aciertos y errores detallados de un modelo de clasificación.
from sklearn.metrics import precision_score	Mide el porcentaje de aciertos positivos dentro de todo lo que el modelo predijo como positivo.
from sklearn.metrics import recall_score	Mide la proporción de casos positivos reales que el modelo fue capaz de identificar correctamente.
from sklearn.metrics import accuracy_score	Calcula el porcentaje general de predicciones correctas hechas por el modelo de clasificación.
from sklearn.decomposition import PCA	Reduce el número de variables reduciendo la dimensionalidad del dataset sin perder la información más importante.
from sklearn.preprocessing import PolynomialFeatures	Genera nuevas características mediante combinaciones polinómicas para modelar relaciones no lineales.

In [ ]:
```
#lectura del .csv con pandas
data_df = pd.read_csv("/content/drive/MyDrive/Colab
Notebooks/Analitica_De_Datos/U2/Ejemplos/data.csv")
data_df
```

Out[ ]:

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.30010	0.14710
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.08690	0.07017
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.19740	0.12790
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.24140	0.10520
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.19800	0.10430
...	...	...	...	...	...	...	...	...	...	...
564	926424	M	21.56	22.39	142.00	1479.0	0.11100	0.11590	0.24390	0.13890
565	926682	M	20.13	28.25	131.20	1261.0	0.09780	0.10340	0.14400	0.09791
566	926954	M	16.60	28.08	108.30	858.1	0.08455	0.10230	0.09251	0.05302
567	927241	M	20.60	29.33	140.10	1265.0	0.11780	0.27700	0.35140	0.15200
568	92751	B	7.76	24.54	47.92	181.0	0.05263	0.04362	0.00000	0.00000

569 rows × 32 columns

Parte 1. EDA

Haz que el `id` sea el índice del dataframe y efectúa una exploración inicial de los datos a través de:

In [ ]:
```
#utilice funcion .set_index()
#Guarde los cambios con inplace=True

#-------------------------------------------------------
#Escriba su codigo
#-------------------------------------------------------
data_df.set_index('id', inplace=True)
data_df
#-------------------------------------------------------
```

Out[ ]:

id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	symm
842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.30010	0.14710	
842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.08690	0.07017	
84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.19740	0.12790	
84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.24140	0.10520	
84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.19800	0.10430	
...	...	...	...	...	...	...	...	...	...	...
926424	M	21.56	22.39	142.00	1479.0	0.11100	0.11590	0.24390	0.13890	
926682	M	20.13	28.25	131.20	1261.0	0.09780	0.10340	0.14400	0.09791	
926954	M	16.60	28.08	108.30	858.1	0.08455	0.10230	0.09251	0.05302	
927241	M	20.60	29.33	140.10	1265.0	0.11780	0.27700	0.35140	0.15200	
92751	B	7.76	24.54	47.92	181.0	0.05263	0.04362	0.00000	0.00000	

569 rows × 31 columns

En esta parte se manda como indice a `id` para que el data frame no genere la columna separada que hace siempre que se crear el data frame

In [ ]:
```
#Desgloce de todas las columnas del data frame
data_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 569 entries, 842302 to 92751
Data columns (total 31 columns):
 #   Column                   Non-Null Count  Dtype  
---  ------                   --------------  -----  
 0   diagnosis                569 non-null    object 
 1   radius_mean              569 non-null    float64
 2   texture_mean             569 non-null    float64
 3   perimeter_mean           569 non-null    float64
 4   area_mean                569 non-null    float64
 5   smoothness_mean          569 non-null    float64
 6   compactness_mean         569 non-null    float64
 7   concavity_mean           569 non-null    float64
 8   concave points_mean      569 non-null    float64
 9   symmetry_mean            569 non-null    float64
 10  fractal_dimension_mean   569 non-null    float64
 11  radius_se                569 non-null    float64
 12  texture_se               569 non-null    float64
 13  perimeter_se             569 non-null    float64
 14  area_se                  569 non-null    float64
 15  smoothness_se            569 non-null    float64
 16  compactness_se           569 non-null    float64
 17  concavity_se             569 non-null    float64
 18  concave points_se        569 non-null    float64
 19  symmetry_se              569 non-null    float64
 20  fractal_dimension_se     569 non-null    float64
 21  radius_worst             569 non-null    float64
 22  texture_worst            569 non-null    float64
 23  perimeter_worst          569 non-null    float64
 24  area_worst               569 non-null    float64
 25  smoothness_worst         569 non-null    float64
 26  compactness_worst        569 non-null    float64
 27  concavity_worst          569 non-null    float64
 28  concave points_worst     569 non-null    float64
 29  symmetry_worst           569 non-null    float64
 30  fractal_dimension_worst  569 non-null    float64
dtypes: float64(30), object(1)
memory usage: 142.2+ KB
```

Desglosamos todas las columnas con `.info()` para saber su tipo de dato, como se ve en su mayoria son de tipo `float64` y solo una de tipo `object` que es la de **"diagnosis"**

In [ ]:
```
#Determinar la cantidad de valores unicos por columna
data_df.nunique()
```

Out[ ]:

	0
diagnosis	2
radius_mean	456
texture_mean	479
perimeter_mean	522
area_mean	539
smoothness_mean	474
compactness_mean	537
concavity_mean	537
concave points_mean	542
symmetry_mean	432
fractal_dimension_mean	499
radius_se	540
texture_se	519
perimeter_se	533
area_se	528
smoothness_se	547
compactness_se	541
concavity_se	533
concave points_se	507
symmetry_se	498
fractal_dimension_se	545
radius_worst	457
texture_worst	511
perimeter_worst	514
area_worst	544
smoothness_worst	411
compactness_worst	529
concavity_worst	539
concave points_worst	492
symmetry_worst	500
fractal_dimension_worst	535

dtype: int64

Listamos los valores unicos por cada una de las columnas con `.nunique()`

In [ ]:
```
data_df.describe().T
```

Out[ ]:

	count	mean	std	min	25%	50%	75%	max
radius_mean	569.0	14.127292	3.524049	6.981000	11.700000	13.370000	15.780000	28.11000
texture_mean	569.0	19.289649	4.301036	9.710000	16.170000	18.840000	21.800000	39.28000
perimeter_mean	569.0	91.969033	24.298981	43.790000	75.170000	86.240000	104.100000	188.50000
area_mean	569.0	654.889104	351.914129	143.500000	420.300000	551.100000	782.700000	2501.00000
smoothness_mean	569.0	0.096360	0.014064	0.052630	0.086370	0.095870	0.105300	0.16340
compactness_mean	569.0	0.104341	0.052813	0.019380	0.064920	0.092630	0.130400	0.34540
concavity_mean	569.0	0.088799	0.079720	0.000000	0.029560	0.061540	0.130700	0.42680
concave points_mean	569.0	0.048919	0.038803	0.000000	0.020310	0.033500	0.074000	0.20120
symmetry_mean	569.0	0.181162	0.027414	0.106000	0.161900	0.179200	0.195700	0.30400
fractal_dimension_mean	569.0	0.062798	0.007060	0.049960	0.057700	0.061540	0.066120	0.09744
radius_se	569.0	0.405172	0.277313	0.111500	0.232400	0.324200	0.478900	2.87300
texture_se	569.0	1.216853	0.551648	0.360200	0.833900	1.108000	1.474000	4.88500
perimeter_se	569.0	2.866059	2.021855	0.757000	1.606000	2.287000	3.357000	21.98000
area_se	569.0	40.337079	45.491006	6.802000	17.850000	24.530000	45.190000	542.20000
smoothness_se	569.0	0.007041	0.003003	0.001713	0.005169	0.006380	0.008146	0.03113
compactness_se	569.0	0.025478	0.017908	0.002252	0.013080	0.020450	0.032450	0.13540
concavity_se	569.0	0.031894	0.030186	0.000000	0.015090	0.025890	0.042050	0.39600
concave points_se	569.0	0.011796	0.006170	0.000000	0.007638	0.010930	0.014710	0.05279
symmetry_se	569.0	0.020542	0.008266	0.007882	0.015160	0.018730	0.023480	0.07895
fractal_dimension_se	569.0	0.003795	0.002646	0.000895	0.002248	0.003187	0.004558	0.02984
radius_worst	569.0	16.269190	4.833242	7.930000	13.010000	14.970000	18.790000	36.04000
texture_worst	569.0	25.677223	6.146258	12.020000	21.080000	25.410000	29.720000	49.54000
perimeter_worst	569.0	107.261213	33.602542	50.410000	84.110000	97.660000	125.400000	251.20000
area_worst	569.0	880.583128	569.356993	185.200000	515.300000	686.500000	1084.000000	4254.00000
smoothness_worst	569.0	0.132369	0.022832	0.071170	0.116600	0.131300	0.146000	0.22260
compactness_worst	569.0	0.254265	0.157336	0.027290	0.147200	0.211900	0.339100	1.05800
concavity_worst	569.0	0.272188	0.208624	0.000000	0.114500	0.226700	0.382900	1.25200
concave points_worst	569.0	0.114606	0.065732	0.000000	0.064930	0.099930	0.161400	0.29100
symmetry_worst	569.0	0.290076	0.061867	0.156500	0.250400	0.282200	0.317900	0.66380
fractal_dimension_worst	569.0	0.083946	0.018061	0.055040	0.071460	0.080040	0.092080	0.20750

Se describen cada una de las columnas con `.describe()` y utilizamos `.T` para trasponer osea cambiamos columnas por filas y viceversa

1a) Estadísticas descriptivas para todas las variables del dataframe.

In [ ]:
```
#.describe para variables Numericas
#-----------------------------------------------------
data_df.describe(include='number').T
#-----------------------------------------------------
```

Out[ ]:

	count	mean	std	min	25%	50%	75%	max
radius_mean	569.0	14.127292	3.524049	6.981000	11.700000	13.370000	15.780000	28.11000
texture_mean	569.0	19.289649	4.301036	9.710000	16.170000	18.840000	21.800000	39.28000
perimeter_mean	569.0	91.969033	24.298981	43.790000	75.170000	86.240000	104.100000	188.50000
area_mean	569.0	654.889104	351.914129	143.500000	420.300000	551.100000	782.700000	2501.00000
smoothness_mean	569.0	0.096360	0.014064	0.052630	0.086370	0.095870	0.105300	0.16340
compactness_mean	569.0	0.104341	0.052813	0.019380	0.064920	0.092630	0.130400	0.34540
concavity_mean	569.0	0.088799	0.079720	0.000000	0.029560	0.061540	0.130700	0.42680
concave points_mean	569.0	0.048919	0.038803	0.000000	0.020310	0.033500	0.074000	0.20120
symmetry_mean	569.0	0.181162	0.027414	0.106000	0.161900	0.179200	0.195700	0.30400
fractal_dimension_mean	569.0	0.062798	0.007060	0.049960	0.057700	0.061540	0.066120	0.09744
radius_se	569.0	0.405172	0.277313	0.111500	0.232400	0.324200	0.478900	2.87300
texture_se	569.0	1.216853	0.551648	0.360200	0.833900	1.108000	1.474000	4.88500
perimeter_se	569.0	2.866059	2.021855	0.757000	1.606000	2.287000	3.357000	21.98000
area_se	569.0	40.337079	45.491006	6.802000	17.850000	24.530000	45.190000	542.20000
smoothness_se	569.0	0.007041	0.003003	0.001713	0.005169	0.006380	0.008146	0.03113
compactness_se	569.0	0.025478	0.017908	0.002252	0.013080	0.020450	0.032450	0.13540
concavity_se	569.0	0.031894	0.030186	0.000000	0.015090	0.025890	0.042050	0.39600
concave points_se	569.0	0.011796	0.006170	0.000000	0.007638	0.010930	0.014710	0.05279
symmetry_se	569.0	0.020542	0.008266	0.007882	0.015160	0.018730	0.023480	0.07895
fractal_dimension_se	569.0	0.003795	0.002646	0.000895	0.002248	0.003187	0.004558	0.02984
radius_worst	569.0	16.269190	4.833242	7.930000	13.010000	14.970000	18.790000	36.04000
texture_worst	569.0	25.677223	6.146258	12.020000	21.080000	25.410000	29.720000	49.54000
perimeter_worst	569.0	107.261213	33.602542	50.410000	84.110000	97.660000	125.400000	251.20000
area_worst	569.0	880.583128	569.356993	185.200000	515.300000	686.500000	1084.000000	4254.00000
smoothness_worst	569.0	0.132369	0.022832	0.071170	0.116600	0.131300	0.146000	0.22260
compactness_worst	569.0	0.254265	0.157336	0.027290	0.147200	0.211900	0.339100	1.05800
concavity_worst	569.0	0.272188	0.208624	0.000000	0.114500	0.226700	0.382900	1.25200
concave points_worst	569.0	0.114606	0.065732	0.000000	0.064930	0.099930	0.161400	0.29100
symmetry_worst	569.0	0.290076	0.061867	0.156500	0.250400	0.282200	0.317900	0.66380
fractal_dimension_worst	569.0	0.083946	0.018061	0.055040	0.071460	0.080040	0.092080	0.20750

Aplicamos lo de antes pero ahora solo para las variables numericas con `include=number` dentro de `.describe()`

In [ ]:
```
#.describe() para variable categoricas
#-------------------------------------------------------
data_df.describe(include=['object','bool', 'category']).T
#-------------------------------------------------------
```

Out[ ]:

	count	unique	top	freq
diagnosis	569	2	B	357

Se hace lo mismo que antes pero con la diferencia de un arreglo dentro de `include` que busca variables categoricas de tipo `object, bool, category`

1b) Valores únicos por variable para identificar posibles variables categóricas.

In [ ]:
```
# Se hace una lista de variables numéricas y otra de categóricas.
#-----------------------------------------------------
num_cols = data_df.select_dtypes(include=np.number).columns.tolist() #para variables numericas
cat_cols = data_df.select_dtypes(exclude=np.number).columns.tolist() #para variables Categoricas
#-----------------------------------------------------
```

In [ ]:
```
# Recuento de cada categoría única en las variables categóricas
for column in cat_cols:
    print(column)
    print(data_df[column].value_counts())
    print('-' * 50)

#Forma numero 2, en este caso como solo es una se puede hacer asi tambien
print("Forma #2")
print("Recuento de cada categoria unica de las variables categoricas")
data_df['diagnosis'].value_counts()
```

```
diagnosis
diagnosis
B    357
M    212
Name: count, dtype: int64
--------------------------------------------------
Forma #2
Recuento de cada categoria unica de las variables categoricas
```

Out[ ]:

diagnosis	count
B	357
M	212

dtype: int64

Utilizamos la primera forma con la cual utilizando la variable `cat_cols` que es para la lista de las variables categoricas, dentro de un ciclo `for` el cual itera sobre cada columan de las variables categoricas, y cuenta cada uno de los valores que este tiene, en este caso solo son dos **B** y **M**. Hay una segunda forma para este caso la cual es solo con `data_df['diagnosis'].value_counts()` esta se puede realizar gracias a que solo es una variable categorica entre todas las del data frame.

1c) Búsqueda de valores faltantes.

In [ ]:
```
#Explica cada linea de codigo
print('Número valores faltantes: ') #imprime en pantalla un mensaje
for col in data_df.columns: #iteracion for entre cada columna del data frame
  valor=sum(data_df[col].isna()) #suma cuantos valores faltan dentro de cada columna
  porcentaje=data_df[col].isna().mean()*100 #calcula el porcentaje total de valores faltantes por cada columna
  print('{}:{}: porcentaje :{}'.format(col,valor,porcentaje)) #arroja el resultado completo de valores y porcentajes faltantes por cada una de las columnas
```

```
Número valores faltantes:
diagnosis:0: porcentaje :0.0
radius_mean:0: porcentaje :0.0
texture_mean:0: porcentaje :0.0
perimeter_mean:0: porcentaje :0.0
area_mean:0: porcentaje :0.0
smoothness_mean:0: porcentaje :0.0
compactness_mean:0: porcentaje :0.0
concavity_mean:0: porcentaje :0.0
concave points_mean:0: porcentaje :0.0
symmetry_mean:0: porcentaje :0.0
fractal_dimension_mean:0: porcentaje :0.0
radius_se:0: porcentaje :0.0
texture_se:0: porcentaje :0.0
perimeter_se:0: porcentaje :0.0
area_se:0: porcentaje :0.0
smoothness_se:0: porcentaje :0.0
compactness_se:0: porcentaje :0.0
concavity_se:0: porcentaje :0.0
concave points_se:0: porcentaje :0.0
symmetry_se:0: porcentaje :0.0
fractal_dimension_se:0: porcentaje :0.0
radius_worst:0: porcentaje :0.0
texture_worst:0: porcentaje :0.0
perimeter_worst:0: porcentaje :0.0
area_worst:0: porcentaje :0.0
smoothness_worst:0: porcentaje :0.0
compactness_worst:0: porcentaje :0.0
concavity_worst:0: porcentaje :0.0
concave points_worst:0: porcentaje :0.0
symmetry_worst:0: porcentaje :0.0
fractal_dimension_worst:0: porcentaje :0.0
```

Con esto podemos ver que las variables numericas estan completas no tienen ningun valor faltante y su porcentaje de igual forma es `0.0`, por lo tanto los datos estan completos y no hace falta usar nada para complementarlos.

1d) Diagrama de barras para determinar la frecuencia de los diagnósticos (cantidad de observaciones con resultado benigno y maligno)

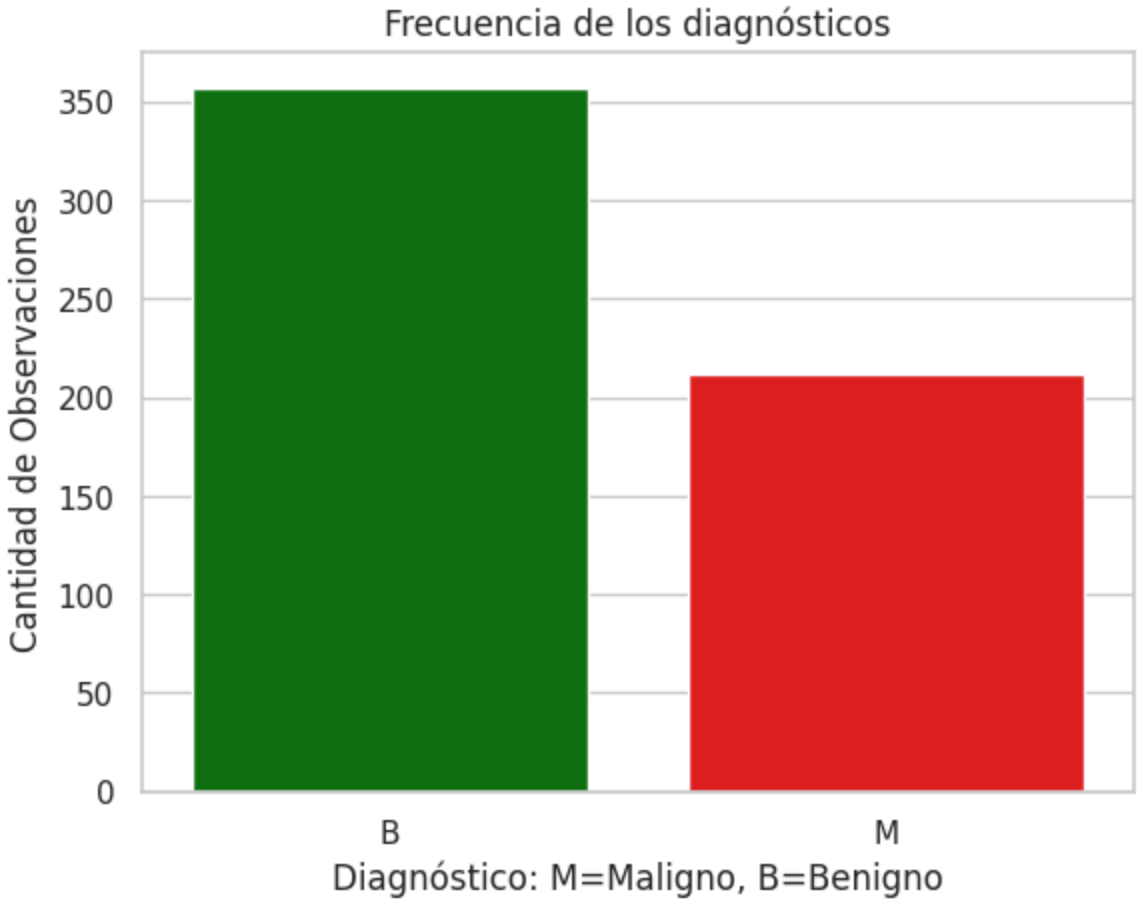
In [ ]:
```python
#utilice sns.countplot()
#-----------------------------------------------------
sns.set_theme(style="whitegrid")
sns.countplot(data=data_df, x='diagnosis', palette=['green', 'red'], order=['B', 'M'])
plt.title('Frecuencia de los diagnósticos')
plt.xlabel('Diagnóstico: M=Maligno, B=Benigno')
plt.ylabel('Cantidad de Observaciones')
#-----------------------------------------------------
```

```
/tmp/ipykernel_4421/3326737552.py:4: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0.
Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.countplot(data=data_df, x='diagnosis', palette=['green', 'red'], order=['B', 'M'])
```

Out[ ]: Text(0, 0.5, 'Cantidad de Observaciones')



La variable `diagnosis` que seria el **diagnostico** tiene dos casos los cuales son Malignos: M, y Beningnos: B.

Mediante la realizacion de la grafica podemos contemplar si los dos casos son iguales o diferentes, pero como se muestra que las el recuento de `Beningno: "B"`, es mucho mayor a los `Malignos: "M"` en este caso.

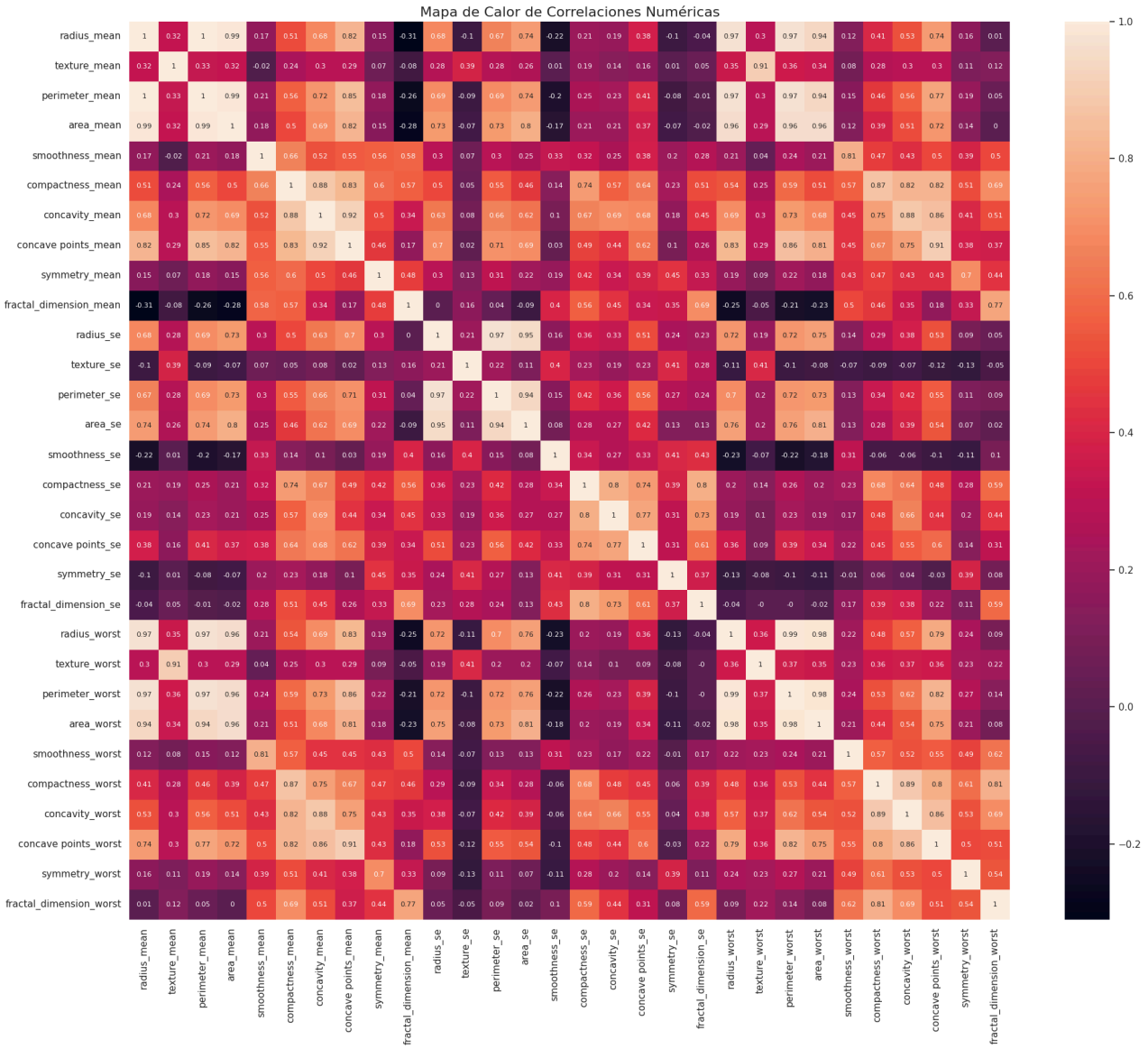
2. Como hay tres valores relacionados con la misma característica (`mean`, `se` y `worst`) es muy probable que exista multicolinealidad en el conjunto.

La multicolinealidad en regresión es una condición que ocurre cuando algunas variables predictoras están fuertemente correlacionadas entre sí, de tal manera que si se incluyen simultáneamente en un modelo, impiden explicar de manera correcta el efecto que cada una tiene sobre la variable respuesta. Existen muchas formas de analizar si hay colinealidad en los modelos, una de ellas es el alto coeficiente de correlación entre variables.

Para observar este efecto, elabora un mapa de calor que cuantifique la correlación de las variables numéricas en el dataframe.

In [ ]:
```
# esta variable es para seleccionar solos los datos de tipo float e int del data frame
numeric_df = data_df.select_dtypes(include=[float, int])

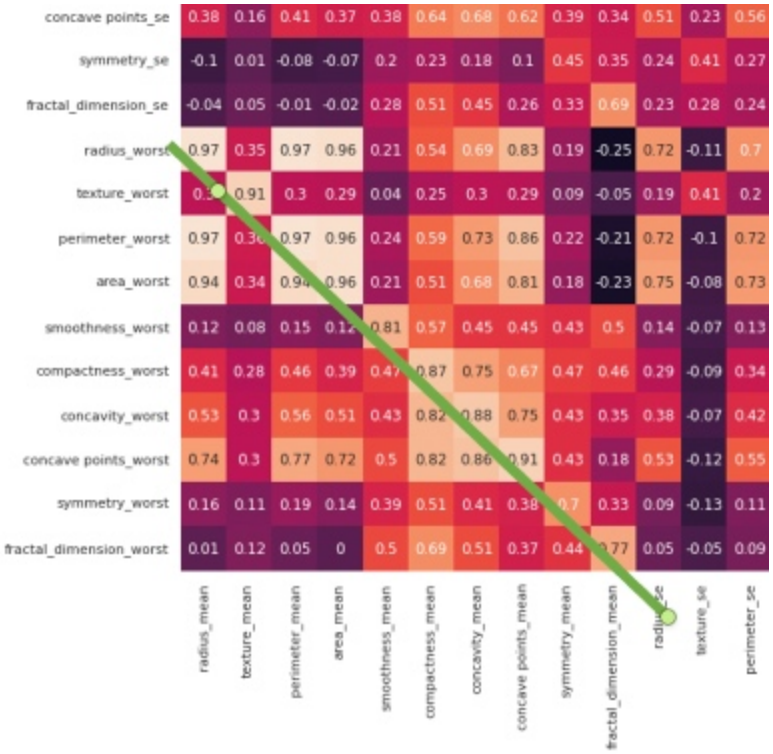
#-------------------------------------------------------
plt.figure(figsize=(20, 17)) #COnfiguramos una medida mayor para que el diagrma salga bien
sns.heatmap(round(numeric_df.corr(numeric_only=True), 2), annot=True, cmap='rocket',
annot_kws={'size': 8}) # aqui desplegamos el diagrama con solo las variables numericas del data frame
plt.title('Mapa de Calor de Correlaciones Numéricas', fontsize=16) #titulo del diagrama
plt.tight_layout()
#-------------------------------------------------------
```



Se puede observar por el diagrama que hay distintas variables las cuales tienen una relacion muy fuerte, fuera del centro que es 1, hay variables dispersas con relacion fuerte y ademas de eso una

segunda columna la cual esta muy relacionada em forma diagonal, de `radus_worst` hasta `fractal_dimension_mean` que va de entre `0.70 - 0.97`.

Si te fijas en los valores de correlación entre las variables `_mean` y `_worst` es evidente la multicolinearidad.



Por ejemplo, la columna `radius_mean` tiene una correlación de 0.97 con la columna `radius_worst`. Esto es algo inevitable, porque las columnas "peores" son esencialmente solo un subconjunto de las columnas "medias".

Para solucionar el problema numérico de la multicolinealidad, tradicionalmente se recurre a eliminar variables o efectuar análisis de componentes principales (PCA) con las `X`'s y usar los componentes como variables independientes en un modelo final.

Conduciremos esta actividad en esos dos sentidos.

Parte 2. Modelo con eliminación de variables altamente correlacionadas

Elimina las variables altamente correlacionadas:

3a) Ahora que sabes que las variables `_mean` y `_worst` tienen correlación alta, hay que quitar del dataframe un conjunto. Borra las columnas `_worst`.

In [ ]:
```
#-----------------------------------------------------
#Elimine TODAS las columnas _worst

#-----------------------------------------------------

#para borrar las columnas _worst
for col in data_df.columns:
  if '_worst' in col:
    data_df.drop(col, axis=1, inplace=True)

#-----------------------------------------------------
#Convierte new_data_df en Dataframe usando .DataFrame() de la libreria de pandas

new_data_df=pd.DataFrame(data_df)
new_data_df

#-----------------------------------------------------
```

Out[ ]:

id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	symm
842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.30010	0.14710	
842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.08690	0.07017	
84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.19740	0.12790	
84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.24140	0.10520	
84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.19800	0.10430	
...	...	...	...	...	...	...	...	...	...	
926424	M	21.56	22.39	142.00	1479.0	0.11100	0.11590	0.24390	0.13890	
926682	M	20.13	28.25	131.20	1261.0	0.09780	0.10340	0.14400	0.09791	
926954	M	16.60	28.08	108.30	858.1	0.08455	0.10230	0.09251	0.05302	
927241	M	20.60	29.33	140.10	1265.0	0.11780	0.27700	0.35140	0.15200	
92751	B	7.76	24.54	47.92	181.0	0.05263	0.04362	0.00000	0.00000	

569 rows × 21 columns

En las graficas de dispersion, los patrones lineales en los diagramas de dispersion se refieren a la relacion entre dos variables que sigue una tendencia aproximadamente en linea recta, ya sea en direccion ascendente o descentende

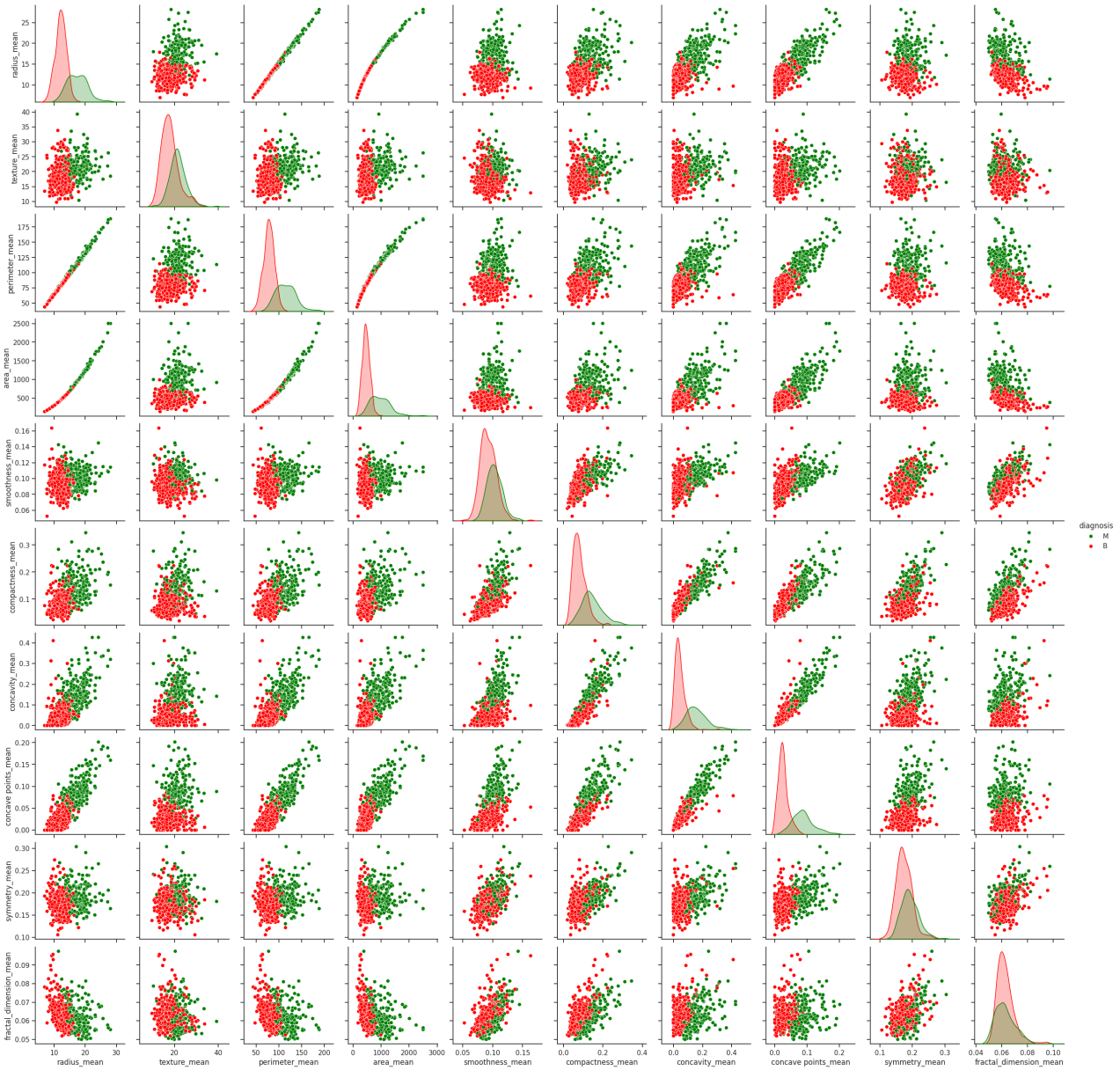
- **Patron lineal positivo**: Si la linea se inclina hacia arriba (de izquierda a derecha), indica que hay una relacion directa entre las variables: cuando la variable aumenta, la otra tambien lo hace, es decir, CORRELACION POSITIVA
- **Patron lineal negativo**: si la linea se incluna hacia abajo (de izquierda a derecha), indica quue cuando una variable aumenta, la otra disminuye, es decir, CORRELACION NEGATIVA
- **Patron lineal perfecto**, si los puntos siguen una linea recta perfecta, la relacion entre las variables es exactamente lineal, sin desviaciones.

3b. Entre las variables `_mean`, identifica patrones lineales con diagramas de dispersión usando:

```
    sns.pairplot(data=nuevo_df[['radius_mean',
        'texture_mean',
        'perimeter_mean',
        'area_mean',
        'smoothness_mean',
        'compactness_mean',
        'concavity_mean',
        'concave points_mean',
        'symmetry_mean',
        'fractal_dimension_mean']])
```

In [ ]:
```
#-----------------------------------------------------
#Pega aqui el codigo sns.pairplot()
sns.set_theme(style='ticks')
sns.pairplot(
    data_df[['radius_mean',
             'texture_mean',
             'perimeter_mean',
             'area_mean',
             'smoothness_mean',
             'compactness_mean',
             'concavity_mean',
             'concave points_mean',
             'symmetry_mean',
             'fractal_dimension_mean',
             'diagnosis']],
    hue='diagnosis',
    palette=['green', 'red']
)
#-----------------------------------------------------
```

Out[ ]:
```
<seaborn.axisgrid.PairGrid at 0x7fda1c123230>
```



Se puede observar claramente en las graficas de dispersion que tienen una relacion muy fuerte entre las variables `readius_mean` , `perimeter_mean` y `area_mean` , las cuales forman o se acercan a un patron lineal perfecto en la que las dos variables aumentan de forma constante y en igualdad.

Las otras variables que casi llegan a un patron lineal pero son un patron lineal positivo son `compactness_mean` , `concativy_mean` y `concave points_mean` ya que su dibujo marca una tendencia acendente muy marcada.

De la matriz podrás observar relaciones lineales bastante evidentes entre:

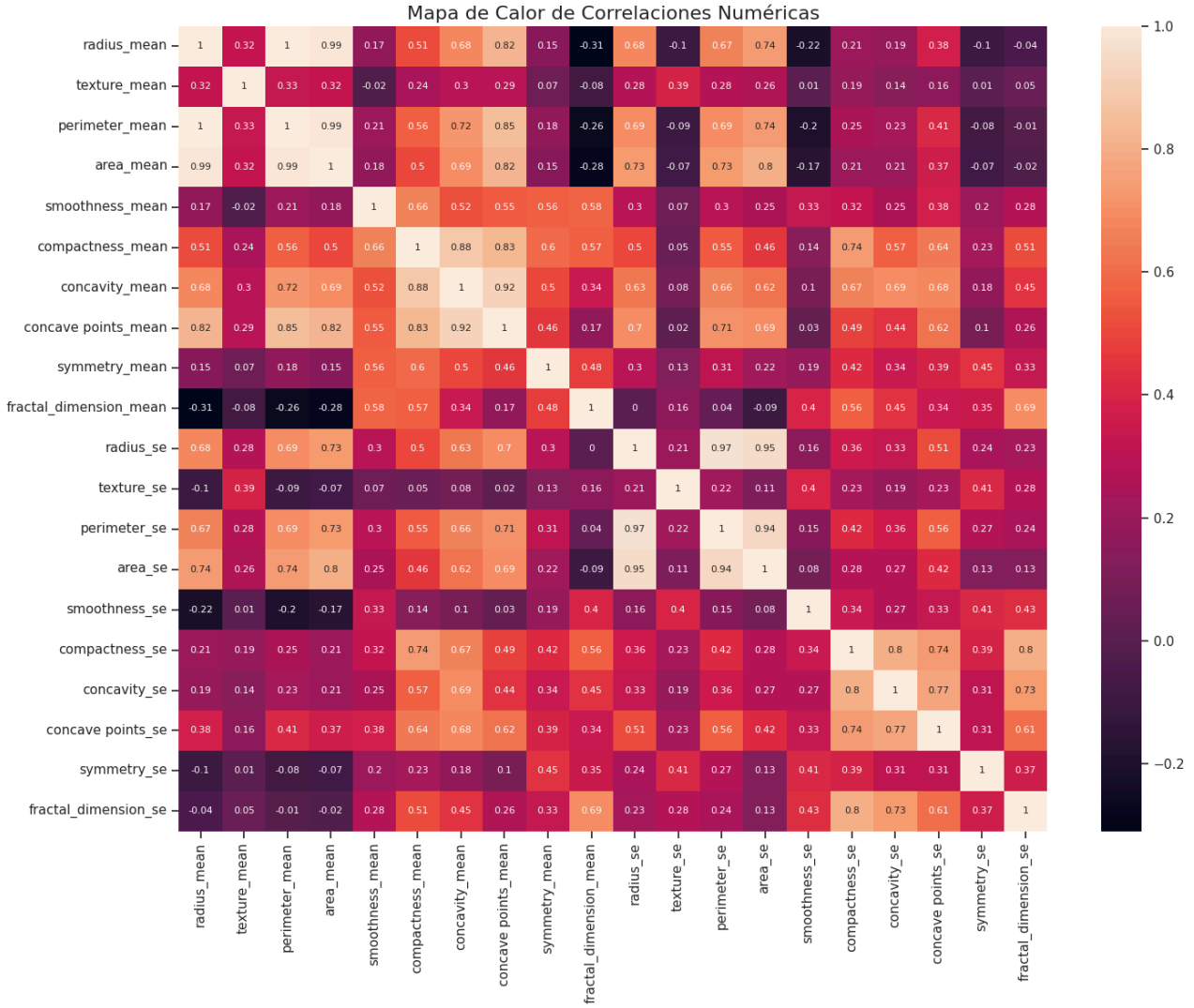
- `radius_mean` , `perimeter_mean` y `area_mean`
- `compactness_mean` , `concavity_mean` , `concave_points_mean`

Sabemos que el perímetro y el área de un círculo, se calculan a partir del radio. Entonces, la relación entre las primeras tres variables es muy clara para nosotros.

3c) Elabora otro mapa de calor confirmar con los valores de correlación.

In [ ]:
```
#-----------------------------------------------------
#Agrega codigo para el mapa de calor
new_numeric_df = new_data_df.select_dtypes(include=[float, int])

#-----------------------------------------------------
plt.figure(figsize=(15, 12))
sns.heatmap(round(new_numeric_df.corr(numeric_only=True), 2), annot=True,
cmap='rocket', annot_kws={'size': 8})
plt.title('Mapa de Calor de Correlaciones Numéricas', fontsize=16)
plt.tight_layout()
#-----------------------------------------------------
```



3d) Después de observar los valores, nos quedaremos con sólo una variable de cada trío: `radius_mean` y `compactness_mean`. Elimina las restantes, no sólo del conjunto `_mean`, sino también de `_se`.

In [ ]:
```
#-----------------------------------------------------
# escriba TODAS las columnas que seran eliminadas (corregido ortografía)
```

```
columnas_a_eliminar = [
  'perimeter_mean',
  'area_mean',
  'concavity_mean',
  'concave points_mean',
  'perimeter_se',
  'area_se',
  'concavity_se',
  'concave points_se'
]
#-----------------------------------------------------

#-----------------------------------------------------
new_data_df_2 = pd.DataFrame(new_data_df.drop(columns=columnas_a_eliminar))
new_data_df_2
#-----------------------------------------------------
```

Out[ ]:

id	diagnosis	radius_mean	texture_mean	smoothness_mean	compactness_mean	symmetry_mean	fractal_dimension_mean	radius_se	texture_se	s
842302	M	17.99	10.38	0.11840	0.27760	0.2419	0.07871	1.0950	0.9053	
842517	M	20.57	17.77	0.08474	0.07864	0.1812	0.05667	0.5435	0.7339	
84300903	M	19.69	21.25	0.10960	0.15990	0.2069	0.05999	0.7456	0.7869	
84348301	M	11.42	20.38	0.14250	0.28390	0.2597	0.09744	0.4956	1.1560	
84358402	M	20.29	14.34	0.10030	0.13280	0.1809	0.05883	0.7572	0.7813	
...	...	...	...	...	...	...	...	...	...	...
926424	M	21.56	22.39	0.11100	0.11590	0.1726	0.05623	1.1760	1.2560	
926682	M	20.13	28.25	0.09780	0.10340	0.1752	0.05533	0.7655	2.4630	
926954	M	16.60	28.08	0.08455	0.10230	0.1590	0.05648	0.4564	1.0750	
927241	M	20.60	29.33	0.11780	0.27700	0.2397	0.07016	0.7260	1.5950	
92751	B	7.76	24.54	0.05263	0.04362	0.1587	0.05884	0.3857	1.4280	

569 rows × 13 columns

Observa la distribución de las variables resultantes (deben ser 12, sin contar la columna de indice):

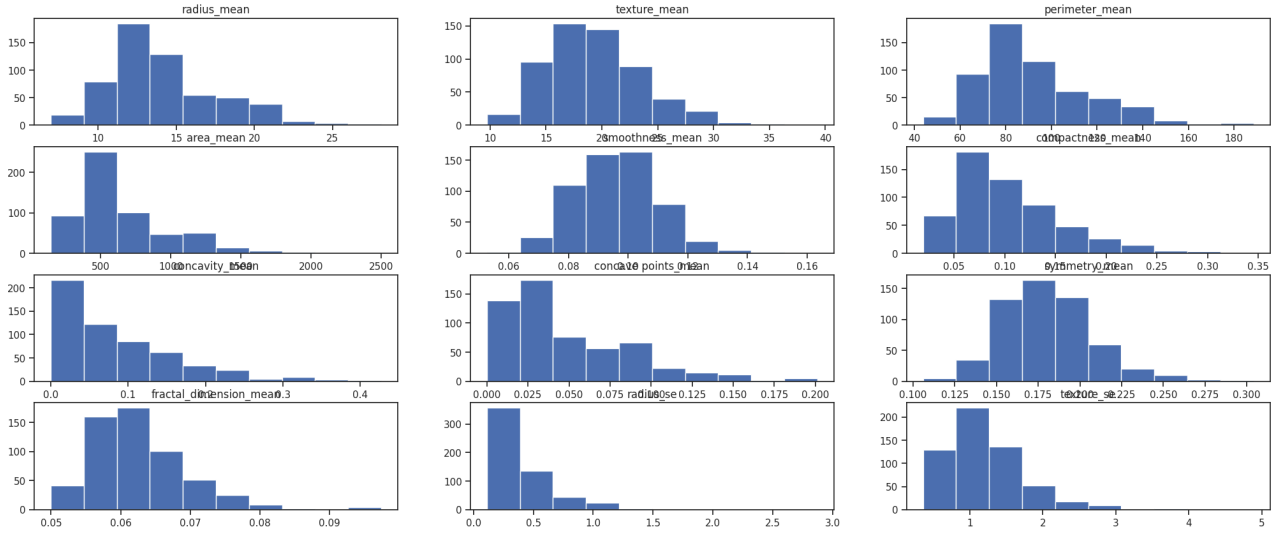
4a) Utilizando histogramas. Guarda en una variable (`skew_cols`) las que tengan marcado sesgo positivo. Para dar seguridad a tu selección, elige aquellas cuyo resultado de aplicar la función `skew()` sea mayor a 1.

In [ ]:

```
#EXPLICA CADA LINEA DE CODIGO
skew_cols_df=pd.DataFrame() # crea un data frame vacio
fig, axes =plt.subplots(4,3, figsize=(25,10)) #crea un lienzo para las graficas asi
como
# medidas de los histogragamas asi como las columnas y filas en las que aparecera cada
uno
axes = axes.ravel() # esto la convierte en un arreglo de 12 elementos gracias a
.reveal()
num_cols = new_data_df.select_dtypes(include=np.number).columns.tolist() # con esto
solo seleccionamos las columnas que cuentan con variables numericas
```

```
for col, ax in zip (num_cols, axes):
  ax.hist(new_data_df[col])
  ax.set(title = f'{col}', xlabel=None)
  if new_data_df[col].skew()>1 : skew_cols_df[col] =new_data_df[col]
# con esto primero iteramos en cada columna asi como espacio del grafico
# luego dibuja un histograma dentro de cada espacio de ax
# anade el titulo dinamicamente a cada uno de los graficos
# evalua la asimetria de los datos de esa columna,  luego si la condicion anterior
cumple copia la columna en el data frame vacio

skew_cols = skew_cols_df.columns.values
# esta parte extre unicamente y guarda en el arreglo los nombres de las columnas con
sesgos muy altos
```



In [ ]:
```
skew_cols # aqui represnetamos los nombres de las columnas con un sesgo muy alto
```

Out[ ]:
```
array(['area_mean', 'compactness_mean', 'concavity_mean',
       'concave points_mean', 'fractal_dimension_mean', 'radius_se',
       'texture_se'], dtype=object)
```

4b) Dibujando box plots de todas las variables. Guarda en una variable (`scale_cols`) aquellas que no se encuentren en el intervalo [0,1]

In [ ]:
```
scale_cols_df = pd.DataFrame()
fig, axes = plt.subplots(4, 3, figsize=(25, 15))
axes = axes.ravel()

# Obtenemos las columnas numéricas de nuestro DataFrame filtrado (new_data_df_2)
num_cols_filtered = new_data_df_2.select_dtypes(include=np.number).columns.tolist()

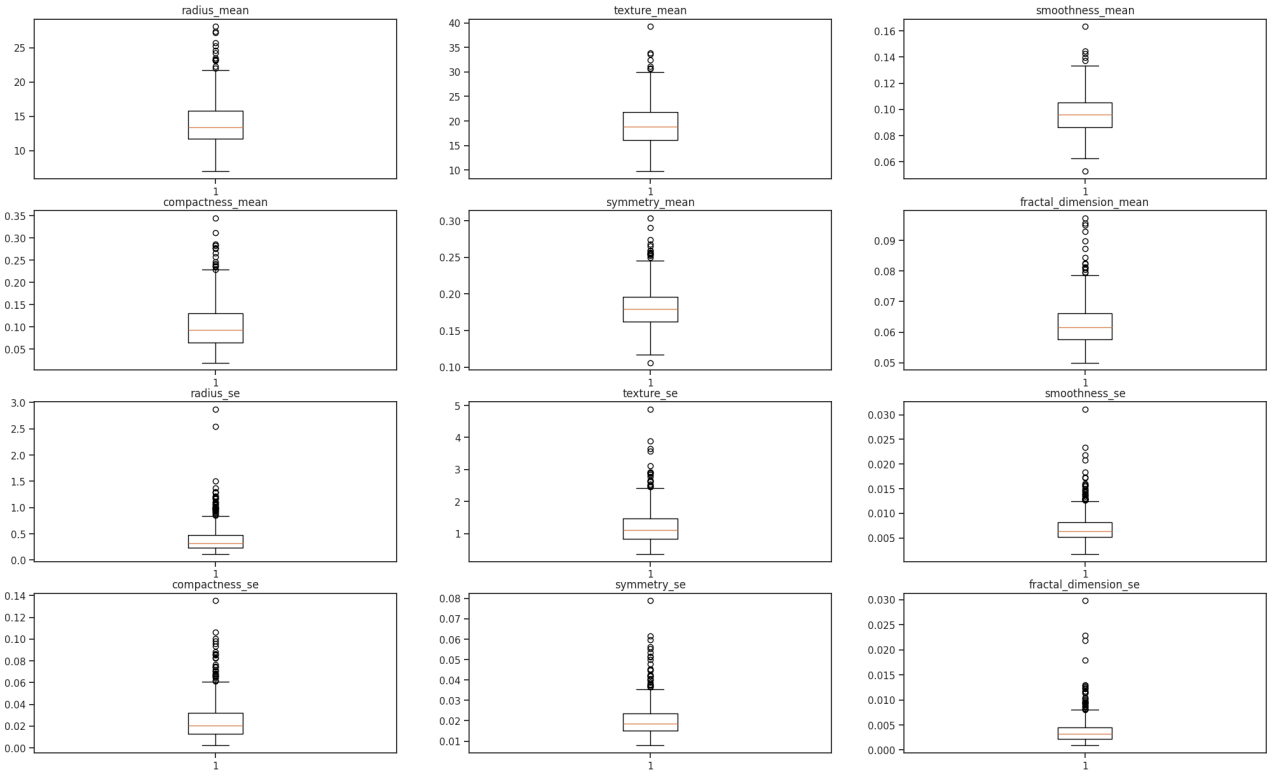
#-----------------------------------------------------
for col, ax in zip(num_cols_filtered, axes):

  # 1. Dibujamos el boxplot pasando la columna correspondiente de datos
  ax.boxplot(new_data_df_2[col])
  ax.set_title(col)

  # 2. Identificamos variables que NO estén contenidas puramente en el intervalo [0,
```

```
1]
  # Es decir, si su máximo es mayor a 1 o su mínimo es menor a 0
  if new_data_df_2[col].max() > 1 or new_data_df_2[col].min() < 0:
    scale_cols_df[col] = new_data_df_2[col]
#-----------------------------------------------------

scale_cols = scale_cols_df.columns.values
```



In [ ]:
```
scale_cols
```

Out[ ]:
```
array(['radius_mean', 'texture_mean', 'radius_se', 'texture_se'],
      dtype=object)
```

Con todo el análisis anterior, estamos listos para generar un modelo *baseline* denominado `logr_model`. Para ello:

5a) Vuelve a leer el contenido del archivo, haz que el `id` sea el índice y separa las variables del dataframe: en `X` coloca los predictores y en `y` la variable de respuesta o salida (`diagnosis`). Divide el conjunto en entrenamiento y prueba (80:20) considerando el parámetro `random_state` con el valor de 1.

In [ ]:
```
data_df = pd.read_csv('/content/drive/MyDrive/Colab
Notebooks/Analitica_De_Datos/U2/Ejemplos/data.csv')

#-----------------------------------------------------
#AGREGA LINEA DE CODIGO PARA CONVERTIR COLUMNA "ID" EN INDICE
data_df = data_df.set_index('id')
#-----------------------------------------------------
```

In [ ]:
```
X = data_df.copy() #Se crea copia del df


#-----------------------------------------------------
#                                  COMPLETA CODIGO
# se elimina variable Categorica "diagnosis" de (X), debido a se sera utilizada como
variable de salida (y)
#-----------------------------------------------------
#Especifique la columna que desea eliminar
#Utilice el parametro axis=1, permitira eliminar la columna
#Guarde los cambios agregando el parametro inplace=True

X.drop('diagnosis', axis=1, inplace=True) #Caracteristica / variable Entrada
#-----------------------------------------------------

#-----------------------------------------------------
y = data_df['diagnosis'] #Objetivo /Variable salida
#-----------------------------------------------------
```

In [ ]:
```
# Librería necesaria para realizar la partición del conjunto de datos
from sklearn.model_selection import train_test_split

# Se divide el conjunto en entrenamiento y prueba (80:20), es decir, 80% para
Entrenamiento (train) y 20% para Prueba (test)
Xtrain, Xtest, ytrain, ytest = train_test_split(X, y, test_size=0.2, random_state=1)
```

5b) Prepara un transformador, denominado `preprocessing`, para borrar las columnas altamente correlacionadas (las 18 variables que se determinaron en los ejercicios previos) Asegúrate de incluir el parámetro `remainder='passthrough'` para mantener el resto de las variables.

In [ ]:
```
#EXPLIQUE CADA LINEA DE CODIGO
columnas_a_eliminar=
['radius_worst','texture_worst','perimeter_worst','area_worst','smoothness_worst','com
pactness_worst','concavity_worst','concave
points_worst','symmetry_worst','fractal_dimension_worst','perimeter_mean','area_mean',
'concavity_mean','concave
points_mean','perimeter_se','area_se','concavity_se','concave points_se']
#En esta linea anterior se muestra un listado de las columnas que se necesitan
eliminar para las columnas que estan alatamente relacionadas

#Este paso esta diseñado exclusivamente para aplicar pasos especificos en las columnas
que se seleccionaron
preprocessing = ColumnTransformer([
    ('num', 'drop', columnas_a_eliminar)] #Se indica que aplique drop a las columnas
que se decidieron fueran eliminadas
   ,remainder='passthrough') #Esto asegura que las columnas restantes de los datos
queden intactas sin ser modificadas o eliminadas
```

5c) Entrena el modelo `logr_model` con el transformador `preprocessing` y regresión logística.

Como la salida `y` está en términos de las etiquetas `'B'` y `'M'`, en lugar de 0 y 1, para evaluar el modelo en el conjunto de prueba deberás especificar la clase positiva. En el caso de la matriz de confusión, indica el orden de las etiquetas con `labels=['B','M']`, porque `'B'` es la clase negativa (ésta se especifica primero) y `'M'` la positiva. Para las métricas de *recall* y *precision*, utiliza el parámetro `pos_label='M'`. Como *accuracy* ocupa la suma de ambas clases y el total de las observaciones, no requiere ninguna especificación. Si quisieras omitir estos parámetros, tendrías que sustituir `'B'` por 0 y `'M'` por 1, previo a la construcción del modelo.

In [ ]:
```
# Creamos la 'tubería' o flujo de trabajo primero aplica nuestro filtro para borrar
las columnas repetidas y luego entrenar el modelo
logr_model = make_pipeline(preprocessing, LogisticRegression())

# Entrenamos el modelo usando nuestros datos de práctica de entrada y sus respuestas
reales
logr_model.fit(Xtrain, ytrain)

# Le pedimos al modelo que intente adivinar los diagnósticos del conjunto de examen o
prueba
predictions = logr_model.predict(Xtest)

# Creamos una matriz de confusiónque compara cuantas veces el modelo acerto y cuantas
fallo al predecir
print('matriz de confusion 1:\n', confusion_matrix(ytest, predictions, labels=
['B','M']))

#Calculamos el recall de todos los casos que eran realmente
print('recall    :\t', recall_score(ytest, predictions, pos_label='M'))

# Calculamos la presicion de todos los casos que el modelo clasifico
print('precision :\t', precision_score(ytest, predictions, pos_label='M'))

#Calculamos el acurracy el porcentaje general de respuestas correctas
print('accurancy :\t', accuracy_score(ytest, predictions))
```

```
matriz de confusion 1:
 [[68  4]
 [10 32]]
recall    :      0.7619047619047619
precision :      0.8888888888888888
accurancy :      0.8771929824561403
```

Análisis del Modelo 1 (Baseline)

- **Matriz de Confusión:**
  - **Verdaderos Benignos (B predicho como B):** 68 casos. El modelo es muy bueno identificando tumores benignos.
  - **Falsos Positivos (B predicho como M):** 4 casos. Se clasificaron erróneamente 4 casos benignos como malignos.
  - **Falsos Negativos (M predicho como B):** 10 casos. Esto es lo más peligroso, ya que 10 pacientes con tumores malignos se fueron a casa pensando que estaban sanos.

    - **Verdaderos Malignos (M predicho como M):** 32 casos.
- **Métricas:**
    - **Recall (Sensibilidad): 76.19%** — El modelo logra detectar aproximadamente el 76% de los cánceres reales. Es un nivel un poco bajo para temas médicos donde no podemos dejar pasar casos graves.
    - **Precisión: 88.88%** — Cuando el modelo dice que un tumor es maligno, tiene un 88.8% de probabilidad de tener la razón.
    - **Accuracy (Exactitud): 87.71%** — En general, el modelo acierta casi el 88% de todas sus predicciones.

Para generar un modelo `logr_model2` con transformación y escalamiento:

6a) Modifica el transformador anterior para, además del borrado de las columnas correlacionadas, aplicar la raíz cuadrada a los predictores con sesgo (previamente almacenados en `skew_cols`) y escalamiento *MinMax* a los predictores con escala mayor a 1 (previamente almacenados en `scale_cols`) Como no todos los predictores serán eliminados o transformados, asegúrate de incluir el parámetro `remainder='passthrough'`

In [ ]:
```
#EXPLIQUE CADA LINEA DE CODIGO
preprocessing2 = ColumnTransformer([
    # Esta encargada de eliminar las columnas que se especificaron anteriormente
    ('num0','drop', columnas_a_eliminar),
    # Aplica la transformacionde R² solo a las columnas con un sesgo positivo
    ('num1',FunctionTransformer(np.sqrt),skew_cols),
    # Escala las columnas en un rango de 0,1
    ('num2',MinMaxScaler(),scale_cols)
    # Hace que cualquier otra columna no sea manipulada y queden intactas
  ],remainder='passthrough')
```

6b) Entrena el modelo `logr_model2` con regresión logística, aplicando previamente el transformador modificado. Evalúa el modelo en el conjunto de prueba con las mismas métricas.

In [ ]:
```
# 1. Creamos la tubería utilizando el preprocesamiento transformado (preprocessing2)
logr_model2 = make_pipeline(preprocessing2, LogisticRegression())

# 2. Entrenamos el modelo con el conjunto de entrenamiento
logr_model2.fit(Xtrain, ytrain)

# 3. Realizamos las predicciones en el conjunto de prueba
predictions2 = logr_model2.predict(Xtest)

# 4. Evaluamos con las métricas requeridas
print('matriz de confusion 2:\n', confusion_matrix(ytest, predictions2, labels=
['B','M']))
print('recall    :\t', recall_score(ytest, predictions2, pos_label='M'))
print('precision :\t', precision_score(ytest, predictions2, pos_label='M'))
print('accurancy :\t', accuracy_score(ytest, predictions2))
```

```
matriz de confusion 2:
 [[70  2]
 [ 9 33]]
recall    :      0.7857142857142857
precision :      0.9428571428571428
accurancy :      0.9035087719298246
```

Análisis del Modelo 2

- **Matriz de Confusión:**
  - **Verdaderos Benignos (B predicho como B):** Sube de 68 a 70 casos correctos.
  - **Falsos Positivos (B predicho como M):** Baja de 4 a solo 2 casos.
  - **Falsos Negativos (M predicho como B):** Baja de 10 a 9 casos. Sigue habiendo un margen de mejora, pero es más seguro que el anterior.
  - **Verdaderos Malignos (M predicho como M):** Aumenta de 32 a 33 casos.
- **Métricas:**
  - **Recall: 78.57%** — Subió un 2.4% con respecto al primer modelo, logrando detectar más casos reales de cáncer.
  - **Precisión: 94.28%** — Aumentó significativamente de 88.8% a un 94.2%. Ahora es mucho más confiable cuando da un diagnóstico positivo.
  - **Accuracy: 90.35%** — Superamos la barrera del 90% de exactitud general.

Conclusión de la Práctica

Con esta práctica me quedó mucho más claro por qué no podemos simplemente meter los datos directo a un modelo de machine learning sin antes revisarlos bien.

Al principio, gracias al análisis exploratorio y a los gráficos como el mapa de calor, nos dimos cuenta de que muchas variables estaban súper correlacionadas (multicolinealidad), lo que significa que básicamente le estábamos dando información repetida al modelo (como el perímetro y el área que dependen directamente del radio). Al limpiar estas columnas redundantes, aligeramos el dataset.

Además, vimos que aplicar transformaciones matemáticas (como la raíz cuadrada para quitar el sesgo de los datos) y escalar las variables con `MinMaxScaler` para que todas jueguen bajo las mismas reglas (en un rango de 0 a 1) ayuda muchísimo. Al comparar los dos modelos, el segundo mejoró bastante:

- La exactitud general pasó del 87% al 90%.
- La precisión subió un montón, casi al 94%, lo que significa que el modelo falla mucho menos al asegurar si un tumor es malo.
- El recall también mejoró, algo que en medicina es de vida o muerte para no dejar ir a un paciente enfermo pensando que está sano.

En resumen, aprendí que la clave de un buen modelo no es solo el algoritmo que elijas, sino el tiempo que le dediques a limpiar, transformar y preparar tus datos antes de entrenarlo.